INF563 · VÉRIFICATION ET VALIDATION DE LOGICIEL · MASTER 2 SIGL

Solveur de l'équation du second degré testé avec JUnit 5

Fonctionnement détaillé du projet : théorie, architecture, bibliothèques, annotations, code ligne par ligne, différences entre **SolveurNaifTest** et **SolveurStableTest**, et réponses aux questions de l'enseignant.

84	84/84	18	6
tests exécutés par classe de test	verts contre la version stable	échecs voulus contre la version naïve	familles d'oracles combinées

Mise à jour du 4 novembre 2026

Ce document décrit le projet au stade de la **séance 3 (chapitres I à III)** : ses chiffres (84 tests, 18 échecs) et les sections 3, 6, 7.6, 8 et les annexes ont changé. L'état actuel (interface + réflexion, pairwise, JaCoCo, PIT) est décrit dans **Fonctionnement_detaille_chapitres_4_5.pdf**.

Enseignant : Evaris Fomekong · Niveau visé : chapitres I à III (séance 3 : « Anatomie formelle d'un test et problème de l'oracle »)

Projet : **Projet_test_polynome_chapitre3** · Java 21 · Maven 3.9.11 · JUnit 5.13.4 · AssertJ 3.27.3 · VS Code

Document de compréhension et de préparation à l'oral · novembre 2026

Table des matières

Comment utiliser ce document 4
- Parcours conseillés 4
- Codes visuels 4
- Notations utilisées partout 4

1. Vue d'ensemble du projet 5
- 1.1 Le projet en une phrase 5
- 1.2 D'où vient le sujet ? 5
- 1.3 Les trois idées que le projet démontre 5
- 1.4 Les chiffres clés 5
- 1.5 Correspondance avec les consignes du TP (diapo 83) 6
- 1.6 Ce que l'enseignant a demandé à l'oral (enregistrement du cours) 6

2. Les notions théoriques indispensables 7
- 2.1 L'équation du second degré 7
- 2.2 Les nombres flottants double (norme IEEE 754) 7
- 2.3 Les pièges numériques révélés par le projet 8
- 2.4 La formule stable : pourquoi elle fonctionne 8
- 2.5 La mise à l'échelle par une puissance de 2 9
- 2.6 Conditionnement κ, stabilité et justification des tolérances 9
- 2.7 Le modèle formel d'un test (chapitre III) 10
- 2.8 La mesure d'erreur N 11
- 2.9 Le problème de l'oracle et les familles d'oracles 11

3. L'architecture du projet 13
- 3.1 L'arborescence complète 13
- 3.2 Le rôle de chaque fichier 13
- 3.3 Pourquoi séparer src/main et src/test ? 14
- 3.4 Le paquetage inf563.solveur et les visibilités 14
- 3.5 Le diagramme de classes 14
- 3.6 Comment les fichiers communiquent : le trajet d'un test 15
- 3.7 Le patron de conception : une suite de tests abstraite 15

4. Les bibliothèques et les outils 17
- 4.1 Vue d'ensemble 17
- 4.2 Java 21 : les fonctionnalités du langage utilisées 17
- 4.3 Maven : le chef d'orchestre 17
- 4.4 Le Maven Wrapper (mvnw.cmd) 19
- 4.5 Surefire : le lien entre Maven et JUnit 19
- 4.6 JUnit 5 : architecture et rôle 20
- 4.7 AssertJ : pourquoi une deuxième bibliothèque d'assertions ? 20
- 4.8 La bibliothèque standard Java mobilisée 21
- 4.9 VS Code et l'Extension Pack for Java 21

5. L'importance des annotations « @ » 23
- 5.1 Qu'est-ce qu'une annotation ? 23
- 5.2 Pourquoi JUnit dépend entièrement des annotations 23
- 5.3 Inventaire de toutes les annotations du projet 24
- 5.4 Zoom : @ParameterizedTest et @CsvSource 24
- 5.5 Zoom : @MethodSource et les combinaisons de paramètres 25
- 5.6 Zoom : @TestFactory et les tests dynamiques 25
- 5.7 Zoom : @DisplayName, @Tag et @Override 26
- 5.8 Les annotations JUnit 5 non utilisées que l'enseignant peut citer 26

6. Le code de production expliqué ligne par ligne 27
- 6.1 Solveur.java : la structure de la classe 27
- 6.2 La version naïve : naif(a, b, c) 27
- 6.3 La version stable : stable(a, b, c) 28
- 6.4 Comparaison synthétique des deux versions 28
- 6.5 Demo.java : afficher n'est pas tester 29

7. La suite de tests SuiteOraclesSolveur en détail 30
- 7.1 Plan du fichier 30

7.2 Les éléments communs 30
7.3 Bloc 1 : l'oracle par référence (parReference) 30
7.4 Bloc 2 : le contrat (aNul, discriminantNegatif) 31
7.5 Bloc 3 : les oracles sans résultat attendu 32
7.6 Bloc 4 : toutes les combinaisons de paramètres 32
7.7 Bloc 5 : les 20 tests générés dynamiquement 34
7.8 Bloc 6 : l'oracle différentiel en BigDecimal 34
7.9 Récapitulatif : chaque élément du modèle, test par test 35

8. SolveurNaifTest et SolveurStableTest : la différence 35
8.1 Les deux classes côte à côte 36
8.2 Ce qu'elles ont en commun 36
8.3 Ce qui les différencie 36
8.4 Pourquoi garder une classe de test qui échoue ? 36
8.5 Les 18 échecs de la version naïve, un par un 37
8.6 Le tableau « détecte / ne détecte pas » 37
8.7 Tester une troisième implémentation (celle d'un camarade) 37
8.8 Et si… ? (questions de compréhension) 38

9. Exécuter, lire et déboguer les tests 38
9.1 Ce qui se passe quand on tape .\mvnw.cmd test 39
9.2 Ce que fait le moteur Jupiter pour un test paramétré 39
9.3 Lire les résultats 39
9.4 Dans VS Code, pas à pas 40
9.5 Déboguer un cas qui échoue 40
9.6 Lancer le programme Demo 40
9.7 Commandes utiles (terminal VS Code : Ctrl + ù) 40
9.8 Exercices pour s'approprier le modèle 40
9.9 Dépannage 41
9.10 Le rendu sous Git 41

10. Bilan : résultats, limites et suites possibles 41
10.1 Résultats par bloc de tests 42
10.2 Les limites du projet (à reconnaître honnêtement) 42
10.3 Ce que les chapitres suivants ajouteraient 42

11. Questions probables de l'enseignant et réponses 42
11.1 Le fonctionnement général du projet 43
11.2 Les bibliothèques et les outils 43
11.3 Les annotations « @ » 44
11.4 SolveurNaifTest et SolveurStableTest 45
11.5 Questions techniques : numérique et algorithmes 46
11.6 Les oracles et le chapitre III 46
11.7 Questions d'ouverture et questions pièges 48

12. Glossaire 49

Annexes : le code source complet 50
A. src/main/java/inf563/solveur/Solveur.java 51
B. src/main/java/inf563/solveur/Demo.java 51
C. src/test/java/inf563/solveur/SuiteOraclesSolveur.java 52
D. src/test/java/inf563/solveur/SolveurStableTest.java 52
E. src/test/java/inf563/solveur/SolveurNaifTest.java 55
F. pom.xml 55
G. .vscode/settings.json et .mvn/wrapper/maven-wrapper.properties 55

Comment utiliser ce document

> Ce document explique **tout** le projet, du « pourquoi » jusqu'à la dernière ligne de code. Il est écrit pour que vous puissiez le défendre seul devant l'enseignant : chaque affirmation est vérifiable dans le code ou en relançant les tests, et les valeurs numériques citées viennent d'exécutions réelles sur votre machine (Java 21).

Parcours conseillés

Si vous avez…	Lisez dans cet ordre
30 minutes avant l'oral	Partie 1 (vue d'ensemble), encadrés « À retenir », partie 8 (naïf vs stable), puis la FAQ (partie 11)
Une soirée	Parties 1 à 3 (théorie et architecture), 5 (annotations), 7 (la suite de tests), 8, puis 11
Envie de tout maîtriser	Tout, dans l'ordre, en gardant VS Code ouvert à côté pour relancer chaque test cité

Codes visuels

> **À retenir**
> L'idée essentielle d'une section, à savoir dire en une phrase.

> **Si l'enseignant demande…**
> Une question probable et la réponse attendue, formulée simplement.

> **Piège**
> Une erreur classique (de code ou de raisonnement) et comment l'éviter.

> **Pour aller plus loin**
> Un détail technique utile si la discussion devient pointue.

Notations utilisées partout

Notation	Signification (modèle du chapitre III, diapos 58 à 64)
SC	Scénario / cas de test : la situation précise que l'on vérifie
DE	Données d'entrée : ici le triplet (a, b, c)
g	Système sous test (SUT, *system under test*) : la fonction `racines(a, b, c)`
y	Sortie observée : `y = g(DE)`, le tableau de racines renvoyé
RA	Résultat attendu (la référence), quand on le connaît
N	Mesure d'erreur entre y et RA (ici l'erreur relative `|y − RA| / |RA|`)
Tol	Tolérance : l'écart maximal accepté, justifié par `Tol = c·κ·ε`
O	Oracle : le mécanisme qui décide si y est acceptable (prédicat O(x, y) → vrai/faux)
ε	Epsilon machine : `Math.ulp(1.0)` $= 2^{-52} \approx 2{,}22 \cdot 10^{-16}$
κ	Conditionnement : sensibilité de la racine aux petites perturbations de a, b, c
Δ	Discriminant : `Δ = b² − 4ac`

Les nombres sont écrits à la française dans le texte ($2{,}5 \cdot 10^{-8}$) et à l'anglaise dans le code et les sorties Java (`2.5E-8`). Les numéros de diapositives renvoient au support de cours *INF563_Verification_Validation_Logiciel*.

1. Vue d'ensemble du projet

1.1 Le projet en une phrase

> Le projet **teste automatiquement**, avec la bibliothèque **JUnit 5**, une fonction Java `racines(a, b, c)` qui calcule les racines réelles de l'équation $\mathbf{a \cdot x^2 + b \cdot x + c = 0}$, et montre que la **même suite de tests** accepte une implémentation correcte (formule stable) et **détecte** une implémentation fragile (formule du lycée).

Ce n'est donc pas un projet « qui résout une équation » : c'est un projet **qui vérifie** un programme qui résout une équation. Le cœur du travail est dans les **tests** (dossier `src/test`), pas dans le solveur. C'est exactement l'objet du cours INF563 : la vérification et la validation de logiciel.

1.2 D'où vient le sujet ?

Le solveur du second degré est le **fil rouge A** du cours (diapo 28 : « six fils rouges, un par nature de logiciel ») : il représente les **logiciels de calcul numérique**. Il apparaît dans deux séances :

Séance	Thème	Travail demandé (diapos)
Séance 2 (chap. II)	Fondements machine : entiers, flottants IEEE 754, sources d'erreur	TP « Faire échouer le solveur » (diapos 50 à 54) : écrire la version naïve et la version stable, trouver les entrées qui font échouer la version naïve.
Séance 3 (chap. III)	Anatomie formelle d'un test et problème de l'oracle	TP « Des oracles pour le solveur » (diapo 83) : test paramétré DE / RA / Tol de 8 cas, au moins quatre oracles sans RA, exécution contre les deux versions, tableau « détecte / ne détecte pas », bonus BigDecimal et $\Delta \approx 0$, rendu Git + rapport d'une page.

1.3 Les trois idées que le projet démontre

1. **Un test est une décision, pas un affichage.** Un test qui imprime des racines à l'écran ne décide rien : c'est l'oracle (une assertion JUnit) qui rend un verdict PASS ou FAIL à partir de critères explicites (N, RA, Tol). C'est la diapo 59 (« Un test n'est pas un appel + un affichage ») et c'est aussi ce que l'enseignant a reproché en cours à ceux qui « affichent » leurs résultats.
2. **Un code « mathématiquement juste » peut être numériquement faux.** La formule du lycée `(−b ± √Δ) / 2a` est exacte sur le papier, mais en `double` elle perd 25 % de précision, déborde en $\pm\infty$ ou rend NaN sur des entrées parfaitement légales. Seuls des tests bien conçus le révèlent.
3. **Aucun oracle ne voit tout : il faut les combiner.** Les relations métamorphiques ne détectent pas la version naïve (elle est « fausse mais cohérente ») ; le résidu, Viète, la construction, la référence et le différentiel la détectent. La force de la suite vient de la **combinaison** d'oracles partiels (diapo 84).

1.4 Les chiffres clés

Élément	Valeur	Commentaire
Fichiers Java	5	2 dans `src/main` (Solveur, Demo), 3 dans `src/test` (la suite abstraite + 2 classes concrètes)
Méthodes de test écrites	11	1 `@Test`, 9 `@ParameterizedTest`, 1 `@TestFactory`
Tests exécutés par classe	84	1 + 63 cas paramétrés + 20 tests dynamiques
Implémentations testées	2	stable (`Solveur.racines`) et naïve (`Solveur.racinesNaif`)

Élément	Valeur	Commentaire
Résultat version stable	84 / 84	tout est vert : `BUILD SUCCESS`
Résultat version naïve	66 / 18	66 verts, 18 rouges **voulus** : chaque rouge prouve qu'un oracle détecte un défaut
Durée d'exécution	≈ 0,3 s	pour les 84 tests (hors démarrage de Maven)

1.5 Correspondance avec les consignes du TP (diapo 83)

Consigne de l'enseignant	Où c'est fait dans le projet	État
Reprendre Solveur, versions naïve et stable (séance 2)	`Solveur.naif` et `Solveur.stable`, copiés fidèlement des diapos 51 et 54	fait
Test paramétré DE / RA / Tol de 8 cas, N justifiée	`parReference` : 8 lignes `@CsvSource`, N = erreur relative, Tol = c·κ·ε	fait
Au moins quatre oracles sans RA : résidu, Viète, deux relations métamorphiques	`residu`, `viete`, `echelle`, `oppose` + en plus `construction`, `combinaisons`, `constructionGeneree`	fait (7)
Exécuter la suite contre les deux versions, tableau « détecte / ne détecte pas »	`SolveurStableTest` et `SolveurNaifTest` héritent de la même suite ; tableau dans `RAPPORT.md` et § 8.6	fait
Bonus : test différentiel BigDecimal	`differentiel` (50 chiffres)	fait
Bonus : Δ ≈ 0 avec Tol justifiée par le conditionnement	8e ligne de `parReference` : κ ≈ 1,9·10^{8}, Tol = 5·10^{-7}	fait
Rendu : dépôt Git + rapport d'une page qui justifie chaque tolérance	`RAPPORT.md` ; commandes Git dans `README.md` §7	prêt

1.6 Ce que l'enseignant a demandé à l'oral (enregistrement du cours)

L'enregistrement est très bruité ; seule la fin (≈ 58e à 66e minute) est exploitable. On y entend clairement les attentes suivantes, toutes couvertes par le projet :

Ce qui a été dit (reformulé)	Réponse du projet
Utiliser JUnit (transcrit « jim ») et ses « arobases » `@`	JUnit 5 Jupiter ; 8 annotations différentes, expliquées en partie 5
Afficher des résultats « ce n'est pas comme ça qu'on fait les tests »	Toutes les vérifications sont des assertions ; `Demo` affiche, mais n'est pas un test
Lancer les tests dans VS Code, voir les résultats	Test Explorer (icône fiole) : vert / rouge par cas, message d'échec détaillé
Des cas de test « générés », « dynamiques »	`@TestFactory` : 20 équations générées à l'exécution (graine fixe 563)
Chaque paramètre a son tableau de valeurs et JUnit « combine tous les cas possibles »	`@MethodSource` : 3 valeurs de a × 4 de b × 3 de c = 36 combinaisons
Tester plusieurs paramètres sur un même cas de test (« décorateurs »)	`@ParameterizedTest` + `@CsvSource` : une ligne = un cas
Écrire les tests à partir du **prototype** de la fonction (`double` → liste de `double`), sans connaître son code, pour pouvoir tester la fonction d'un camarade	La suite ne dépend que de `double[] g(double a, double b, double c)` ; brancher le code d'un camarade = une classe de 6 lignes (§ 8.7)

> **À retenir**
>
> Le projet = **une** suite de tests (`SuiteOraclesSolveur`) écrite **une seule fois** contre le prototype `double[] g(double a, double b, double c)`, puis exécutée contre **deux** implémentations grâce à l'héritage. La version stable passe tout ; la version naïve échoue 18 fois, ce qui prouve que les tests savent détecter des défauts numériques réels.

2. Les notions théoriques indispensables

> On ne peut pas expliquer les tests sans comprendre **ce qu'ils cherchent**. Cette partie rassemble les notions des chapitres II (nombres machine) et III (anatomie d'un test, oracles) dont le projet a besoin.

2.1 L'équation du second degré

On cherche les réels x tels que $\mathbf{a{\cdot}x^2 + b{\cdot}x + c = 0}$ avec a ≠ 0. On calcule le discriminant $\mathbf{\Delta = b^2 - 4ac}$:

- $\mathbf{\Delta < 0}$: aucune racine réelle ;
- $\mathbf{\Delta = 0}$: une racine double `x = -b / 2a` ;
- $\mathbf{\Delta > 0}$: deux racines `x = (-b ± √Δ) / 2a` (la « formule du lycée »).

Deux relations, dites **relations de Viète**, relient les racines aux coefficients et servent d'oracles : $\mathbf{r_1 + r_2 = -b/a}$ et $\mathbf{r_1 \cdot r_2 = c/a}$. Si a = 0, ce n'est plus une équation du second degré : le contrat du projet impose alors de lever une `IllegalArgumentException`.

> **Le contrat de la fonction testée (javadoc de Solveur.java)**
>
> `double[] racines(double a, double b, double c)` renvoie : une exception si a = 0 ; un tableau **vide** si Δ < 0 ; un tableau à **une** case si Δ = 0 ; un tableau à **deux** cases triées par ordre croissant si Δ > 0. Toute la suite de tests repose sur ce contrat, et uniquement sur lui.

2.2 Les nombres flottants double (norme IEEE 754)

En Java, `double` est un nombre flottant binaire sur 64 bits : **1 bit de signe**, **11 bits d'exposant**, **52 bits de mantisse** (53 bits de précision avec le bit implicite). Un double vaut ± 1,mantisse × 2^{exposant}. Conséquences directes pour le projet :

Notion	Valeur en double	Pourquoi c'est important ici
Plus grand double fini	`1.797…E308`	b^2 avec b = 10^{200} vaut 10^{400} : **débordement** vers +∞
Plus petit double normalisé / dénormalisé	`2.2E-308` / `4.9E-324`	4ac avec a = c = 10^{-200} vaut $4{\cdot}10^{-400}$: arrondi à **0**
Epsilon machine ε = ulp(1)	$2^{-52} \approx 2{,}22{\cdot}10^{-16}$	unité de mesure des tolérances (`EPS` dans le code)
ulp(x) : écart entre x et le double suivant	$\text{ulp}(10^8) = 2^{-26} \approx 1{,}49{\cdot}10^{-8}$	explique exactement l'erreur de 25 % de la formule naïve (§ 2.3)
Arrondi au plus proche	erreur relative ≤ ε/2 par opération	chaque + − × ÷ √ est presque parfait ; c'est l'**amplification** qui pose problème
Valeurs spéciales	`+∞, -∞, NaN, -0.0`	la formule naïve renvoie `[-Infinity, Infinity]` ou `[NaN, NaN]` sans aucune exception

> **Piège : NaN rend toute comparaison fausse**
>
> `NaN < 0`, `NaN == 0` et `NaN <= tol` valent tous **false** ; `Math.min(NaN, 1)` vaut NaN. C'est pour cela que la version naïve « traverse » ses tests `if (d < 0)` et `if (d == 0)` avec Δ = NaN, et c'est aussi pour cela que le test écrit `assertTrue(n <= tol)` : si N vaut NaN, la condition est fausse, donc le test **échoue** comme il le doit. Écrire `assertFalse(n > tol)` aurait laissé passer NaN (faux négatif).

2.3 Les pièges numériques révélés par le projet

Le tableau suivant est la **trace réelle** de la version naïve, obtenue en exécutant le code sur votre machine. Il reprend les cas de la diapo 52.

DE (a, b, c)	Calculs intermédiaires de la version naïve	Sortie naïve	Attendu	Phénomène
(1, −3, 2)	Δ = 1, √Δ = 1	`[1.0, 2.0]`	[1, 2]	correct
(1, 10^8, 1)	$b^2 = 10^{16}$, Δ = 9.999999999999996E15, √Δ = 9.999999999999999E7, −b + √Δ = **−1.4901161193847656E-8**	`[-1.0E8, -7.450580596923828E-9]`	$[-10^8, -10^{-8}]$	élimination catastrophique : 25 % d'erreur
(1, 10^{200}, 1)	b^2 = **+∞**, Δ = +∞, √Δ = +∞	`[-Infinity, Infinity]`	$[-10^{200}, -10^{-200}]$	débordement d'un calcul intermédiaire
(10^{200}, $-3{\cdot}10^{200}$, $2{\cdot}10^{200}$)	b^2 = +∞, 4ac = +∞, Δ = ∞ − ∞ = **NaN**	`[NaN, NaN]`	[1, 2]	NaN silencieux
(10^{-200}, 1, 10^{-200})	$4ac = 4{\cdot}10^{-400} \to$ **0**, Δ = 1, −b + √Δ = 0	`[-1.0E200, 0.0]`	$[-10^{200}, -10^{-200}]$	absorption : 100 % d'erreur sur la petite racine
(1, −2, $1 - 1{,}1{\cdot}10^{-16}$)	Δ = 4.44E-16 = 2^{-51}, √Δ = 2.107E-8	`[0.99999998946…, 1.00000001054…]`	idem	mal conditionné, mais **juste**

a) L'élimination catastrophique, expliquée au bit près

Pour (1, 10^8, 1), la vraie valeur est $\sqrt{\Delta} = \sqrt{(10^{16} - 4)} \approx 10^8 - 2{\cdot}10^{-8}$. Mais autour de 10^8, deux doubles consécutifs sont espacés de $\text{ulp}(10^8) \approx 1{,}49{\cdot}10^{-8}$: √Δ est arrondi au double le plus proche, $10^8 - 1{,}49{\cdot}10^{-8}$. La soustraction −b + √Δ est ensuite **exacte**, mais elle révèle l'erreur d'arrondi précédente : on obtient $-1{,}49{\cdot}10^{-8}$ (un seul ulp) au lieu de $-2{\cdot}10^{-8}$, d'où $r = -7{,}45{\cdot}10^{-9}$ au lieu de -10^{-8}. Tous les chiffres significatifs ont disparu. Comme le dit la diapo 44 : « la soustraction elle-même est exacte : c'est l'information perdue avant qui est révélée ».

b) Débordement, NaN et absorption

- **Débordement** (*overflow*) : les racines -10^{200} et -10^{-200} sont parfaitement représentables, mais le calcul **intermédiaire** $b^2 = 10^{400}$ ne l'est pas. Résultat : ±∞.
- **NaN silencieux** : ∞ − ∞ n'a pas de valeur, la norme rend NaN. Aucune exception n'est levée ; le NaN se propage jusqu'au résultat. Un test qui n'existe pas ne le verra jamais.
- **Absorption** : $1 - 4{\cdot}10^{-400} = 1$ en double (le petit terme est « absorbé » ; ici il est même d'abord arrondi à 0 par dépassement inférieur). La petite racine vaut alors 0 au lieu de -10^{-200}.

c) Le cas mal conditionné n'est pas un bug

Pour $c = 1 - 1{,}1{\cdot}10^{-16}$ (en double : 0.9999999999999999), changer c d'environ 10^{-16} déplace les racines d'environ 10^{-8}. C'est le **problème** qui est sensible, pas l'algorithme : les deux versions donnent exactement le même résultat, qui est juste à la précision que le problème permet ($\approx \sqrt{\varepsilon} \approx 10^{-8}$). Un bon test doit l'accepter : c'est pourquoi sa tolérance est $5{\cdot}10^{-7}$ et non 10^{-15} (§ 2.6).

2.4 La formule stable : pourquoi elle fonctionne

Dans `(-b ± √Δ) / 2a`, l'un des deux signes additionne deux nombres de **même signe** (sans danger), l'autre soustrait deux nombres **proches** quand $b^2 \gg 4ac$ (élimination). L'idée est de ne calculer que la racine « sans danger », puis d'obtenir l'autre par Viète :

```
q  = −½ · ( b + signe(b)·√Δ )        // b et signe(b)·√Δ ont le même signe : pas de soustraction
r₁ = q / a                           // = (−b − signe(b)·√Δ) / 2a : la racine « sûre »
r₂ = c / q                           // Viète : r₁·r₂ = c/a, donc r₂ = c / (a·r₁) = c / q
```

Vérification que c/q est bien l'autre racine : c/q = −2c / (b + s√Δ). En multipliant en haut et en bas par (b − s√Δ), le dénominateur devient $b^2 - \Delta = 4ac$, d'où c/q = −(b − s√Δ) / 2a = (−b + s√Δ) / 2a. ✓

En Java, `Math.copySign(Math.sqrt(d), b)` renvoie √Δ avec le signe de b : c'est exactement « signe(b)·√Δ ». Sur (1, 10^8, 1), la version stable donne `[-1.0E8, -1.0E-8]` : la petite racine vient de c/q = 1 / (−10^8), une simple division, sans aucune perte.

2.5 La mise à l'échelle par une puissance de 2

La formule stable seule ne règle pas le débordement de b^2. La version stable commence donc par diviser les trois coefficients par une puissance de 2 :

```java
double m = Math.max(Math.abs(a), Math.max(Math.abs(b), Math.abs(c)));   // le plus grand coefficient
int e = Math.getExponent(m);                                            // m = 1,xxx × 2^e
a = Math.scalb(a, -e);  b = Math.scalb(b, -e);  c = Math.scalb(c, -e); // a·2^-e, b·2^-e, c·2^-e
```

- **Les racines ne changent pas** : diviser les trois coefficients par le même nombre non nul donne une équation équivalente.
- **La division est exacte** : multiplier par 2^{-e} ne modifie que l'exposant binaire, pas la mantisse. Aucun arrondi n'est introduit (sauf si un coefficient minuscule devient lui-même trop petit, ce qui le rend négligeable).
- **Plus de débordement** : après mise à l'échelle, le plus grand coefficient est dans [1, 2[, donc $b^2 < 4$ et $|4ac| < 16$.

Trace réelle de la version stable sur les cas difficiles :

DE (a, b, c)	e	Coefficients après mise à l'échelle	Δ'	q	q/a	c/q
(1, 10^8, 1)	26	(1.49E-8, 1.49, 1.49E-8)	2.22	−1.49	`-1.0E8`	`-1.0E-8`
(1, 10^{200}, 1)	664	(1.31E-200, 1.31, 1.31E-200)	1.71	−1.31	`-1.0E200`	`-1.0E-200`
(10^{200}, $-3\cdot10^{200}$, $2\cdot10^{200}$)	665	(0.653, −1.96, 1.31)	0.427	1.31	`2.0`	`1.0`
(10^{-200}, 1, 10^{-200})	0	inchangés	1.0	−1.0	`-1.0E200`	`-1.0E-200`

Le tableau est trié ensuite par `Math.min / Math.max` : la sortie est toujours dans l'ordre croissant.

2.6 Conditionnement κ, stabilité et justification des tolérances

Le cours distingue deux questions (diapo 46) :

- **Conditionnement** (propriété du **problème**) : une petite perturbation des données change-t-elle beaucoup la solution exacte ?
- **Stabilité** (propriété de l'**algorithme**) : l'algorithme ajoute-t-il plus d'erreur que ce que le conditionnement impose ?

Pour une racine r du polynôme $p(x) = ax^2 + bx + c$, si l'on perturbe a, b, c d'une erreur relative δ, la racine bouge de $|\delta r/r| \lesssim \kappa\cdot\delta$ avec :

$$\kappa = \frac{|a|\cdot r^2 + |b|\cdot|r| + |c|}{|r| \cdot |2ar + b|} \qquad (p'(r) = 2ar + b : \text{la pente du polynôme en } r)$$

Un algorithme stable fait au mieux une erreur relative de l'ordre de **κ·ε**. D'où la règle de la diapo 67 : **Tol = c · κ · ε**, avec c ≈ 10 (marge pour quelques arrondis). Valeurs calculées pour les 8 cas de référence :

#	DE (a, b, c)	Diagnostic	κ	10·κ·ε	Tol retenue
1	(1, −3, 2)	cas nominal	6	$1{,}3\cdot10^{-14}$	$2\cdot10^{-14}$
2	(1, 10^8, 1)	élimination catastrophique	2	$4{,}4\cdot10^{-15}$	$5\cdot10^{-15}$
3	(1, 10^{200}, 1)	b^2 déborde en +∞	2	$4{,}4\cdot10^{-15}$	$5\cdot10^{-15}$
4	(10^{200}, $-3\cdot10^{200}$, $2\cdot10^{200}$)	∞ − ∞ = NaN	6	$1{,}3\cdot10^{-14}$	$2\cdot10^{-14}$
5	(10^{-200}, 1, 10^{-200})	absorption	2	$4{,}4\cdot10^{-15}$	$5\cdot10^{-15}$
6	(−2, 3, 5)	a négatif	1,43	$3{,}2\cdot10^{-15}$	$4\cdot10^{-15}$
7	(1, −2, 1)	racine double exacte	∞	—	0
8	(1, −2, 1 − $1{,}1\cdot10^{-16}$)	Δ ≈ 0, mal conditionné	$1{,}9\cdot10^{8}$	$4{,}2\cdot10^{-7}$	$5\cdot10^{-7}$

Exemples de calcul de κ (à savoir refaire au tableau) :

- (1, −3, 2), r = 1 : κ = (1 + 3 + 2) / (1 · |2 − 3|) = **6** ; r = 2 : κ = (4 + 6 + 2) / (2 · |4 − 3|) = **6**.
- (1, 10^8, 1), r ≈ -10^{-8} : κ ≈ (10^{-16} + 1 + 1) / (10^{-8} · 10^8) = **2**. Le problème est très bien conditionné : la formule naïve, qui fait 25 % d'erreur, est donc **instable** (10^{15} fois pire que κ·ε).
- (−2, 3, 5), r = −1 : κ = (2 + 3 + 5) / (1 · |4 + 3|) = 10/7 ≈ **1,43**.
- Racine double (1, −2, 1) : p'(r) = 2r − 2 = 0, donc κ = ∞. Mais ici Δ = 4 − 4 = 0 est calculé **sans aucun arrondi** (petits entiers) et −b/2a = 1 exactement : on peut exiger Tol = 0.
- Δ ≈ 0 : p'(r) ≈ $2{,}1\cdot10^{-8}$, d'où κ ≈ 4 / $2{,}1\cdot10^{-8}$ ≈ $1{,}9\cdot10^{8}$: on ne peut garder qu'environ √ε ≈ 10^{-8} de précision relative.

> **À retenir**
>
> Une tolérance ne se choisit pas « au hasard » : **Tol = c·κ·ε**, écrite et expliquée dans le test. Trop large, elle laisse passer les bugs (faux négatif, le plus dangereux) ; trop étroite, elle crée de faux échecs et l'équipe finit par désactiver le test (diapo 67).

2.7 Le modèle formel d'un test (chapitre III)

Le chapitre III décrit un test avec six éléments (diapo 59) et place l'**oracle** au centre (diapo 61) :

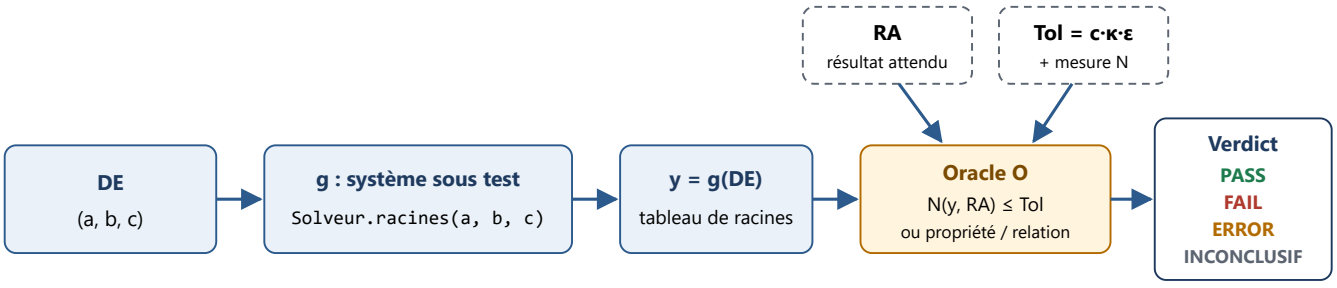


Figure 1 : un test = des données d'entrée, un système sous test, et un oracle qui décide du verdict.

Élément	Rôle	Dans le projet (exemple : élimination catastrophique)
SC	scénario, situation à vérifier	« petite racine de $x^2 + 10^8x + 1$ » : une ligne du `@CsvSource`
DE	valeurs fournies au SUT	(a, b, c) = (1, 1e8, 1) : colonnes 1 à 3
g	ce qu'on exécute	`g(a, b, c)`, c'est-à-dire `Solveur.racines` ou `Solveur.racinesNaif`
RA	sortie de référence	[−99999999.99999999, −1e−8] : colonnes 4 et 5
N	mesure de l'écart	`erreurRelative(y, ra) = |y − RA| / |RA|`

Élément	Rôle	Dans le projet (exemple : élimination catastrophique)
Tol	écart maximal accepté	5e−15 : colonne 6, justifiée par κ = 2
verdict	décision	`assertTrue(n <= tol, …)` → PASS (vert) ou FAIL (rouge)

Le pseudo-code reformulé (diapo 63) précise ce qui se passe pour chaque cas, et c'est exactement ce que fait JUnit :

```
pour chaque cas c = (x, ctx, RA, Tol) de SC :
    préparer(ctx)                       # JUnit : nouvelle instance de la classe de test
    essayer :  y ← g(x)                 # exécution du SUT
               si O(x, y) : PASS  sinon : FAIL           # l'assertion
    si exception e : si e attendue : PASS sinon : ERROR  # assertThrows / exception imprévue
    si délai dépassé : INCONCLUSIF                       # environnement défaillant
    nettoyer(ctx)
verdict_suite ← tous les verdict[c] = PASS              # BUILD SUCCESS ou BUILD FAILURE
```

Les quatre verdicts et leur traduction en JUnit

Verdict	Définition (diapo 64)	Comment JUnit / Maven le montre
PASS	l'oracle accepte, ou l'exception attendue est levée	test vert ✓ (y compris `aNul` grâce à `assertThrows`)
FAIL	l'oracle rejette la sortie : défaut probable du SUT	une assertion lève `AssertionError` → compté dans `Failures`
ERROR	exception imprévue : défaut du SUT ou du test	toute autre exception (ex. `ArrayIndexOutOfBoundsException`) → `Errors`
INCONCLUSIF	délai, base indisponible, réseau : on ne sait pas	hypothèse non remplie (`Assumptions.assumeTrue`) → `Skipped` (non utilisé ici)

2.8 La mesure d'erreur N

La diapo 66 liste les mesures possibles. Le projet utilise l'**erreur relative**, appliquée racine par racine après tri (mesure « ensembles » : on vérifie d'abord le **nombre** de racines, puis on compare élément par élément).

Mesure	Formule	Quand l'utiliser / piège	Dans le projet
Absolue	$\lvert y - \hat{y}\rvert$	grandeurs d'échelle connue ; inadaptée aux très grandes ou très petites valeurs	non (racines de 10^{-200} à 10^{200})
Relative	$\lvert y - \hat{y}\rvert / \lvert\hat{y}\rvert$	calcul flottant, échelles variées ; explose si $\hat{y} \approx 0$	**oui** (aucun RA n'est nul)
Mixte	$\lvert y - \hat{y}\rvert \le atol + rtol\cdot\lvert\hat{y}\rvert$	cas général, deux paramètres à justifier	non
ULP	nb de doubles entre y et ŷ	bibliothèques mathématiques	non
Ensembles	appariement puis N par élément	racines, résultats non ordonnés	**oui** (tri + comparaison case à case)

> **Piège de la diapo 70 : une tolérance absolue masque 25 % d'erreur**
>
> Avec `|y - RA| ≤ tol · max(1, |RA|)` et tol = 10^{-6}, la version naïve **passe** sur (1, 10^{8}, 1) : $|-7{,}45\cdot10^{-9} + 10^{-8}| = 2{,}5\cdot10^{-9} \le 10^{-6}$. Or le résultat est faux de 25 %. Pour |RA| < 1, `max(1, |RA|)` transforme la tolérance relative en tolérance absolue. L'exercice c) du README le fait constater.

2.9 Le problème de l'oracle et les familles d'oracles

Un programme est « non testable » quand aucun oracle n'existe ou qu'il est trop coûteux de décider si une sortie est correcte (Weyuker, 1982, diapo 73). C'est le cas du calcul scientifique : on calcule justement parce

qu'on ne connaît pas la réponse. La solution : remplacer l'oracle complet par plusieurs **oracles partiels**. Le projet en combine **six familles** (taxonomie de Barr et al., 2015, diapo 75) :

Famille	Principe	Tests du projet	Limite
Référence	comparer y à un RA connu : N(y, RA) ≤ Tol	`parReference`	il faut connaître RA (ici calculé à part en haute précision)
Spécifié (contrat)	le comportement imposé par la spécification	`aNul`, `discriminantNegatif`	ne vérifie que les cas prévus par le contrat
Propriété	une propriété que toute bonne sortie vérifie	`residu`, `viete`, `combinaisons`	une propriété n'est qu'une condition nécessaire
Métamorphique	relation entre plusieurs exécutions (T. Y. Chen, 1998)	`echelle`, `oppose`	ne voit pas un programme « faux mais cohérent »
Construction	choisir la réponse, puis fabriquer le problème	`construction`, `constructionGeneree`	$b = -(r_1 + r_2)$ et $c = r_1 r_2$ sont eux-mêmes arrondis
Différentiel	comparer à une autre implémentation	`differentiel` (BigDecimal)	les deux peuvent se tromper de la même façon

À ces six familles s'ajoute l'**oracle implicite** fourni gratuitement par JUnit : toute exception imprévue (plantage) donne le verdict ERROR, même si personne ne l'a demandé.

> **À retenir (diapo 84)**
>
> « Aucun oracle ne voit tout. Une version fausse mais cohérente passe les relations métamorphiques : combiner plusieurs oracles. » Le projet le démontre concrètement : `echelle` et `oppose` restent verts contre la version naïve (et le contrat aussi, puisque les deux versions le respectent), alors que les quatre autres familles (référence, propriété, construction, différentiel) la détectent.

3. L'architecture du projet

> Cette partie répond à la question « quels sont les fichiers, à quoi sert chacun, et comment communiquent-ils ? ».

3.1 L'arborescence complète

```
Projet_test_polynome_chapitre3/
├── pom.xml                              configuration Maven : Java 21, dépendances JUnit 5 + AssertJ, plugins, profils
├── mvnw.cmd, mvnw                       Maven Wrapper (Windows / Linux-macOS) : lance Maven sans l'installer
├── .mvn/wrapper/maven-wrapper.properties    version de Maven à télécharger (3.9.11)
├── .vscode/
│   ├── settings.json                    réglages VS Code du projet (import Maven automatique, wrapper, UTF-8)
│   └── extensions.json                  extension recommandée : Extension Pack for Java
├── .gitignore                           fichiers exclus de Git (target/, audio, PDF du cours)
├── README.md                            guide pas à pas : lancer, déboguer, exercices, dépannage
├── RAPPORT.md                           rapport d'une page demandé (tolérances, tableau détecte / ne détecte pas)
├── doc/                                 ce document
├── src/
│   ├── main/java/inf563/solveur/        CODE DE PRODUCTION (le système sous test)
│   │   ├── Solveur.java                 les deux versions : naif() et stable()
│   │   └── Demo.java                    programme qui affiche le tableau naïf / stable / attendu
│   └── test/java/inf563/solveur/        CODE DE TEST (JUnit 5)
│       ├── SuiteOraclesSolveur.java     LA suite de tests, abstraite, écrite une seule fois
│       ├── SolveurStableTest.java       exécute la suite contre la version stable
│       └── SolveurNaifTest.java         exécute la suite contre la version naïve
└── target/                              (généré) classes compilées et rapports de test Surefire
```

3.2 Le rôle de chaque fichier

Fichier	Nature	Rôle précis	Utilisé par
`Solveur.java`	production	Contient le système sous test : `racines` (publique, appelle `stable`), `racinesNaif` (publique, appelle `naif`), et les deux algorithmes. Ne connaît pas les tests.	les deux classes de test, `Demo`
`Demo.java`	production	Programme avec `main` qui **affiche** le comparatif des deux versions (diapo 52). Sert à **voir**, pas à **vérifier** : ce n'est pas un test.	vous (bouton Run)
`SuiteOraclesSolveur.java`	test (abstrait)	Le cœur du projet : 11 méthodes de test, les données de test, les oracles. Déclare la méthode abstraite `g(a, b, c)` = le système sous test, sans dire lequel.	hérité par les 2 classes concrètes
`SolveurStableTest.java`	test (concret)	Hérite de la suite et dit « g = `Solveur.racines` ». 6 lignes utiles.	JUnit (Maven, VS Code)
`SolveurNaifTest.java`	test (concret)	Hérite de la suite et dit « g = `Solveur.racinesNaif` ». Porte le tag `naif` pour être exclue du build par défaut.	JUnit (profil `naif`, VS Code)
`pom.xml`	build	Décrit le projet pour Maven : identité, version de Java, bibliothèques, plugins, profils `defaut` et `naif`.	Maven, VS Code
`mvnw.cmd` / `mvnw`	build	Scripts du Maven Wrapper : téléchargent Maven 3.9.11 la première fois, puis le lancent.	vous, VS Code
`.mvn/wrapper/…properties`	build	Indique au wrapper quelle version de Maven utiliser et où la télécharger.	`mvnw.cmd`
`.vscode/settings.json`	IDE	Réglages partagés de l'espace de travail (détaillés en § 4.9).	VS Code
`.vscode/extensions.json`	IDE	Recommande l'« Extension Pack for Java » à l'ouverture du dossier.	VS Code
`.gitignore`	Git	Exclut `target/`, les `.class`, l'audio et les PDF du cours du dépôt (`doc/` est réautorisé).	Git

Fichier	Nature	Rôle précis	Utilisé par
`README.md`, `RAPPORT.md`	documentation	Guide d'utilisation et rapport d'une page exigé par le TP.	vous, l'enseignant

3.3 Pourquoi séparer src/main et src/test ?

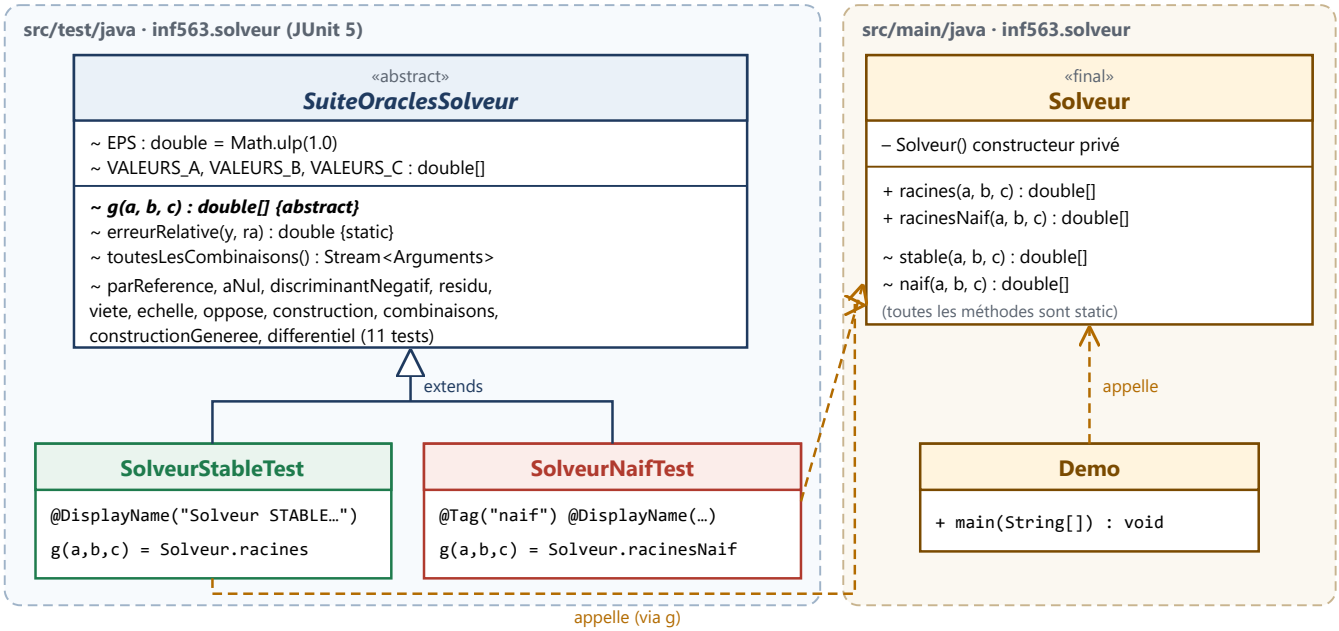
C'est la **convention Maven** (« convention plutôt que configuration ») : Maven sait sans configuration que `src/main/java` contient le code livré et `src/test/java` le code de test. Conséquences :

- le code de test est compilé à part (`target/test-classes`) et **n'est jamais livré** dans le `.jar` ;
- les bibliothèques de test (JUnit, AssertJ, scope `test`) ne sont visibles que depuis `src/test` : le solveur ne peut pas en dépendre par erreur ;
- les deux arborescences utilisent le **même paquetage** `inf563.solveur` : à l'exécution, elles forment un seul paquetage, ce qui permet aux tests d'accéder aux méthodes « paquetage » (`naif`, `stable`) si besoin.

3.4 Le paquetage inf563.solveur et les visibilités

Élément	Visibilité	Justification
`public final class Solveur`	public, final	classe utilitaire : on ne l'étend pas (`final`)
`private Solveur()`	privé	on ne crée jamais d'objet `Solveur` : toutes les méthodes sont `static`
`racines`, `racinesNaif`	public static	le **prototype** exposé : `double[] f(double, double, double)`
`naif`, `stable`	paquetage (aucun mot-clé)	détails d'implémentation, visibles seulement dans `inf563.solveur`
`abstract class SuiteOraclesSolveur`, les deux classes de test, leurs méthodes	paquetage	JUnit 5 n'exige pas `public` (contrairement à JUnit 4) ; seul `private` est interdit

3.5 Le diagramme de classes



Légende : trait plein + triangle = héritage (extends) ; pointillé = appel ; + public, ~ paquetage, – privé ; italique = abstrait.

Figure 2 : diagramme de classes. Les tests dépendent du solveur ; le solveur ne connaît pas les tests.

3.6 Comment les fichiers communiquent : le trajet d'un test

Suivons le test **« élimination catastrophique »** lancé contre la version stable. Personne n'appelle les méthodes de test « à la main » : c'est le **moteur JUnit Jupiter** qui les découvre et les appelle.

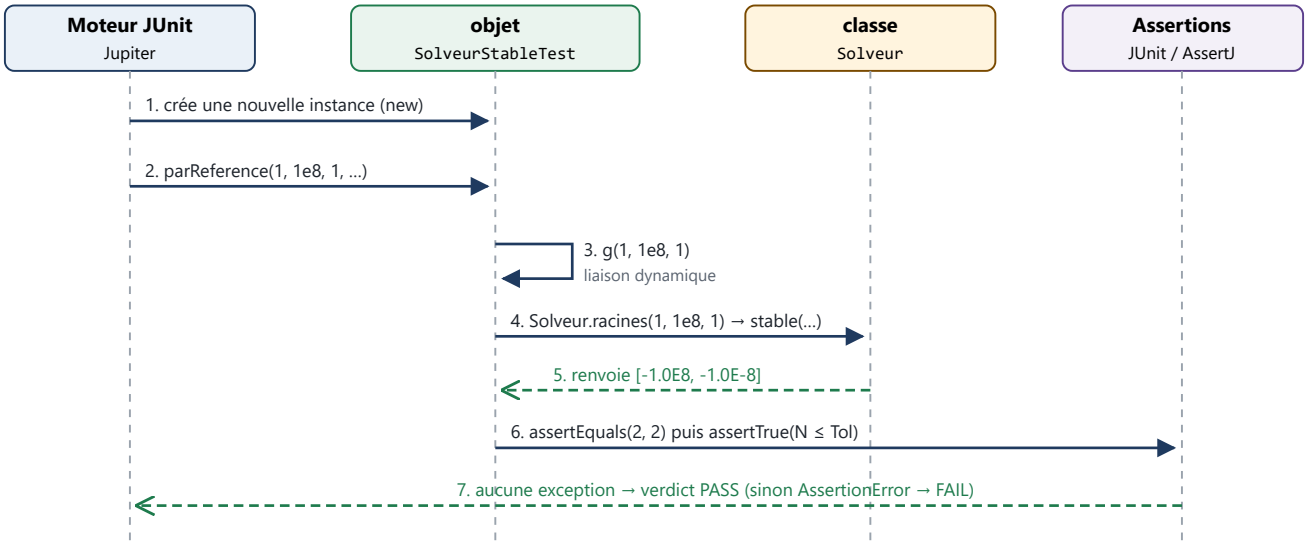


Figure 3 : diagramme de séquence d'un cas de test. Remplacer SolveurStableTest par SolveurNaifTest change seulement l'étape 3-4.

1. **Création** : JUnit crée un objet `SolveurStableTest` neuf pour chaque cas (cycle de vie « par méthode »).
2. **Appel de la méthode de test** : `parReference` n'est pas écrite dans `SolveurStableTest` ; elle est **héritée** de `SuiteOraclesSolveur`. JUnit lui passe les 7 valeurs de la 2e ligne du CSV.
3. **Appel de g** : le code hérité appelle `g(a, b, c)`. Comme l'objet est un `SolveurStableTest`, Java exécute la version de `g` écrite dans cette classe : c'est la **liaison dynamique** (polymorphisme).
4. **Appel du système sous test** : `g` appelle `Solveur.racines`, qui appelle `stable`.
5. **Retour** : le tableau `[-1.0E8, -1.0E-8]` revient dans la méthode de test.
6. **Oracle** : les assertions comparent le nombre de racines puis N ≤ Tol pour chaque racine.
7. **Verdict** : si aucune assertion n'échoue, la méthode se termine normalement → PASS. Sinon une `AssertionError` remonte jusqu'à JUnit → FAIL, avec le message construit dans le test.

3.7 Le patron de conception : une suite de tests abstraite

Le projet applique le patron **Méthode patron** (*Template Method*) aux tests, appelé aussi « test de contrat » ou « suite de tests abstraite » :

- la classe abstraite contient l'**algorithme fixe** (les 11 tests et leurs oracles) ;
- elle délègue une seule étape variable à une méthode abstraite : `g(a, b, c)` ;
- chaque sous-classe concrète fournit cette étape (quelle implémentation tester).

Bénéfice	Explication
Écrire les tests une seule fois	84 tests × 2 implémentations = 168 exécutions, pour un seul fichier de tests.
Comparaison équitable	Les deux versions sont jugées par **exactement** les mêmes données, oracles et tolérances : la différence de résultat vient donc uniquement de l'algorithme.
Tests « boîte noire » sur le prototype	La suite ne connaît que la signature `double[] g(double, double, double)` et le contrat. C'est ce que demandait l'enseignant : écrire le test sans connaître le code.
Extensible (ouvert / fermé)	Tester une 3e implémentation (celle d'un camarade) = ajouter une classe de 6 lignes, sans modifier la suite.

Si l'enseignant demande : « pourquoi g et pas directement Solveur.racines dans les tests ? »

Parce que si les tests appelaient directement `Solveur.racines`, il faudrait **copier** toute la suite pour tester la version naïve (ou celle d'un camarade). Avec `g`, le système sous test devient un **paramètre** de la suite : c'est le « g » du modèle du cours, et l'héritage choisit sa valeur.

Pour aller plus loin : pourquoi SuiteOraclesSolveur n'est jamais exécutée seule

Deux raisons cumulées : (1) elle est `abstract`, donc JUnit ne peut pas l'instancier et l'ignore ; (2) son nom ne correspond pas aux motifs que Surefire recherche par défaut (`*Test`, `Test*`, `*Tests`, `*TestCase`). Ses tests ne s'exécutent qu'à travers les sous-classes concrètes.

4. Les bibliothèques et les outils

> « Jenny », dans la consigne, c'est **JUnit**. Mais un projet de test Java mobilise toute une chaîne d'outils. Cette partie explique chacun : ce qu'il est, pourquoi il est là, et où il est configuré.

4.1 Vue d'ensemble

Outil / bibliothèque	Version	Rôle dans le projet	Où c'est déclaré
JDK Eclipse Temurin	21	compiler (`javac`) et exécuter (JVM) le code Java	installé sur le PC, `JAVA_HOME`
Apache Maven	3.9.11	construire le projet : télécharger les bibliothèques, compiler, lancer les tests	`pom.xml`
Maven Wrapper	3.3.4	télécharger et lancer la bonne version de Maven sans l'installer	`mvnw.cmd`, `.mvn/wrapper/`
maven-compiler-plugin	3.14.0	compiler `src/main` et `src/test` en Java 21	`pom.xml` <build>
maven-surefire-plugin	3.5.3	exécuter les tests JUnit pendant la phase `test`, filtrer par tag, écrire les rapports	`pom.xml` <build>
JUnit 5 (Jupiter)	5.13.4	le **framework de test** : annotations, tests paramétrés et dynamiques, assertions, moteur d'exécution	`pom.xml` (BOM + `junit-jupiter`)
AssertJ	3.27.3	assertions « fluides », en particulier la comparaison **relative** `isCloseTo(…, withinPercentage(…))`	`pom.xml`
Bibliothèque standard Java	21	`Math`, `BigDecimal`, `Random`, `Stream` : calculs, référence haute précision, génération	incluse dans le JDK
VS Code + Extension Pack for Java	—	éditer, compiler en continu, lancer et déboguer les tests (Test Explorer)	`.vscode/`

4.2 Java 21 : les fonctionnalités du langage utilisées

Fonctionnalité	Depuis	Où dans le projet
Blocs de texte `""" … """`	Java 15	le tableau CSV multi-ligne de `parReference` (`@CsvSource(textBlock = …)`)
Expressions lambda `() -> …`	Java 8	`assertThrows(…, () -> g(0, 2, 1))`, corps des tests dynamiques
Streams (`Stream`, `IntStream`, `flatMap`)	Java 8	produit cartésien des combinaisons, génération des 20 tests dynamiques
`Random.nextInt(origine, borne)`	Java 17	tirage des exposants dans `constructionGeneree`
`BigDecimal.sqrt(MathContext)`	Java 9	racine carrée à 50 chiffres dans `differentiel`
`Math.scalb`, `getExponent`, `copySign`, `ulp`	Java 6	mise à l'échelle et formule stable ; ε dans les tests

La propriété `maven.compiler.release = 21` du `pom.xml` garantit que le code est compilé pour Java 21, avec uniquement l'API de Java 21, quelle que soit la version du JDK installée (si elle est ≥ 21).

4.3 Maven : le chef d'orchestre

Maven est un **outil de construction** (*build tool*). À partir d'un fichier de description, le `pom.xml` (*Project Object Model*), il sait télécharger les bibliothèques, compiler, tester et empaqueter. Il fonctionne par **cycle de vie** : demander une phase exécute toutes les phases précédentes.

```
validate → compile → test-compile → test → package → verify → install → deploy
           (src/main)  (src/test)   (Surefire : JUnit)
```

Ainsi `mvnw test` compile d'abord le code, puis les tests, puis les exécute. Les bibliothèques sont identifiées par leurs **coordonnées GAV** (*groupId : artifactId : version*), téléchargées depuis Maven Central puis gardées en cache dans `%USERPROFILE%\.m2\repository` (Internet n'est nécessaire que la première fois).

Le pom.xml expliqué bloc par bloc

```
<groupId>inf563</groupId>                       <!-- G : l'organisation / le cours -->
<artifactId>solveur-second-degre</artifactId>   <!-- A : le nom du projet -->
<version>1.0</version>                          <!-- V : sa version -->
<packaging>jar</packaging>

<properties>
    <maven.compiler.release>21</maven.compiler.release>        <!-- Java 21 -->
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  <!-- accents -->
    <junit.version>5.13.4</junit.version>
    <assertj.version>3.27.3</assertj.version>
</properties>
```

- **Les propriétés** centralisent les versions et réglages. `UTF-8` est indispensable : les noms affichés (« élimination », « Δ ≈ 0 », « ∞ − ∞ ») contiennent des caractères non ASCII.

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.junit</groupId>
            <artifactId>junit-bom</artifactId>     <!-- « Bill Of Materials » -->
            <version>${junit.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

- **Le BOM JUnit** (*Bill Of Materials*) est une « liste de versions compatibles » : il fixe d'un coup la version de tous les modules JUnit (API, paramètres, moteur, plateforme). C'est pourquoi la dépendance `junit-jupiter` ci-dessous n'a **pas** de balise `<version>` : elle vient du BOM. On évite ainsi de mélanger des modules de versions différentes.

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>      <!-- api + params + engine -->
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.assertj</groupId>
        <artifactId>assertj-core</artifactId>
        <version>${assertj.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

- `junit-jupiter` est un **agrégateur** : il apporte `junit-jupiter-api` (annotations, `Assertions`), `junit-jupiter-params` (tests paramétrés) et `junit-jupiter-engine` (le moteur qui exécute).
- `scope test` : ces bibliothèques ne sont disponibles que pour compiler et exécuter les tests ; elles ne font pas partie du programme livré.

```
<plugin>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.5.3</version>
    <configuration>
        <excludedGroups>${groupes.exclus}</excludedGroups>   <!-- tags JUnit à exclure -->
        <groups>${groupes.inclus}</groups>                   <!-- tags JUnit à garder -->
    </configuration>
</plugin>
...
<profile><id>defaut</id>                                      <!-- actif par défaut -->
    <activation><activeByDefault>true</activeByDefault></activation>
    <properties><groupes.exclus>naif</groupes.exclus><groupes.inclus></groupes.inclus></properties>
</profile>
<profile><id>naif</id>                                        <!-- mvnw test -Pnaif -->
    <properties><groupes.exclus></groupes.exclus><groupes.inclus>naif</groupes.inclus></properties>
</profile>
```

- **Les profils** sont des variantes de configuration. Par défaut, le profil `defaut` exclut le tag `naif` : seule `SolveurStableTest` s'exécute et le build est **vert** (indispensable pour une intégration continue). Avec `-Pnaif`, le profil `naif` est activé ; selon la règle Maven, un profil `activeByDefault` est alors désactivé : seuls les tests tagués `naif` s'exécutent.
- Les propriétés `groupes.exclus` / `groupes.inclus` sont des **variables** définies par les profils et injectées dans la configuration de Surefire ; une valeur vide signifie « pas de filtre ».

4.4 Le Maven Wrapper (mvnw.cmd)

Le wrapper permet de lancer Maven **sans l'installer** : `mvnw.cmd` lit `.mvn/wrapper/maven-wrapper.properties`, télécharge Maven 3.9.11 la première fois dans `%USERPROFILE%\.m2\wrapper\dists`, puis le lance avec vos arguments. Tout le monde (vous, l'enseignant, un serveur d'intégration continue) utilise ainsi **la même version** de Maven.

```
wrapperVersion=3.3.4
distributionType=only-script        # pas de fichier .jar : le script fait tout
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3.9.11-bin.zip
```

> **Piège rencontré : l'espace dans « SmartCamp PC1 »**
>
> La version 3.3.2 du wrapper échouait (« 'C:\Users\SmartCamp' n'est pas reconnu… ») car votre dossier utilisateur contient un espace. Le wrapper a été régénéré en version **3.3.4**, dont le script met le chemin entre guillemets.

4.5 Surefire : le lien entre Maven et JUnit

Maven ne sait pas exécuter des tests lui-même : il délègue à **Surefire**. Pendant la phase `test`, Surefire :

1. cherche les classes de test dans `target/test-classes` selon des motifs de nom (`**/*Test.java`, `**/Test*.java`, `**/*Tests.java`, `**/*TestCase.java`) : il trouve `SolveurStableTest` et `SolveurNaifTest` ;
2. détecte JUnit 5 dans les dépendances et utilise le fournisseur **JUnit Platform** ;
3. transmet le filtre de tags (`groups` / `excludedGroups`) ;
4. affiche le bilan (`Tests run: 84, Failures: 0, Errors: 0, Skipped: 0`) et écrit les rapports dans `target/surefire-reports/` (un `.txt` lisible et un `TEST-*.xml` pour les outils) ;
5. fait échouer le build (`BUILD FAILURE`) s'il y a au moins un FAIL ou un ERROR.

4.6 JUnit 5 : architecture et rôle

JUnit 5 n'est pas une seule bibliothèque mais trois sous-projets : **JUnit 5 = JUnit Platform + JUnit Jupiter + JUnit Vintage**.

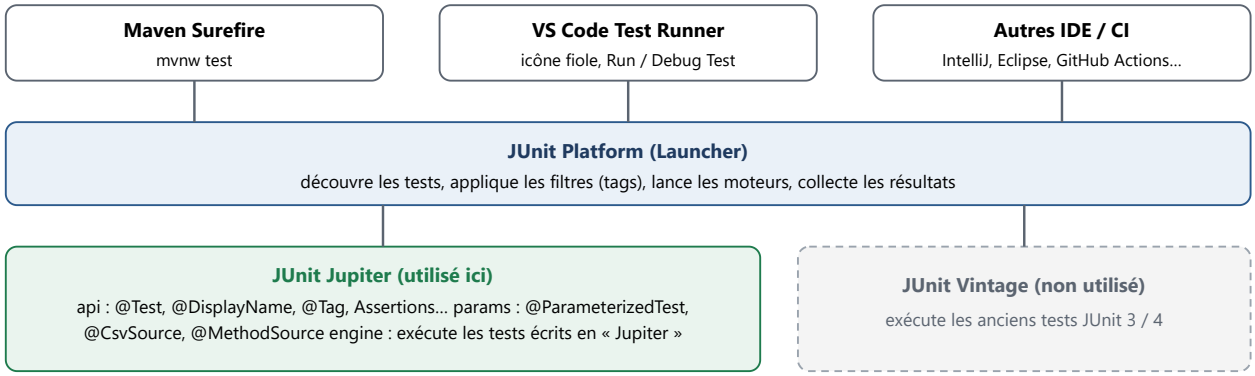


Figure 4 : l'architecture de JUnit 5. Les outils parlent à la Plateforme, qui délègue aux moteurs.

- **JUnit Platform** : le socle commun. Les outils (Surefire, VS Code) ne parlent qu'à elle, via son *Launcher*. C'est elle qui applique le filtre de tags.
- **JUnit Jupiter** : le modèle de programmation de JUnit 5 (les annotations `@…` et les assertions) et son moteur d'exécution. Tout le projet est écrit en Jupiter.
- **JUnit Vintage** : un moteur de compatibilité pour JUnit 3/4. Non utilisé.

Le cycle de vie d'une classe de test Jupiter

Par défaut (`@TestInstance(PER_METHOD)`), JUnit crée **une nouvelle instance** de la classe de test pour chaque test, y compris pour chaque cas d'un test paramétré. Les tests ne partagent donc aucun état : l'ordre d'exécution n'a pas d'importance et un test ne peut pas en « polluer » un autre. Conséquence technique : la méthode qui fournit les combinaisons (`toutesLesCombinaisons`) doit être `static`, car JUnit l'appelle avant de créer les instances.

Les assertions JUnit utilisées

Assertion	Ce qu'elle vérifie	Utilisée dans
`assertEquals(attendu, obtenu, "message")`	égalité (ici sur des `int` : nombre de racines)	parReference, discriminantNegatif, combinaisons
`assertEquals(attendu, obtenu, 0.0)`	égalité de doubles avec un delta **absolu** (ici 0 : égalité stricte)	oppose
`assertTrue(condition, "message")`	une condition booléenne (N ≤ Tol, résidu ≤ borne)	parReference, residu, combinaisons
`assertThrows(Classe.class, () -> …)`	que le code lève bien l'exception attendue	aNul
`assertArrayEquals(t1, t2)`	deux tableaux identiques case à case (égalité stricte des doubles)	echelle

En cas d'échec, ces assertions lèvent `org.opentest4j.AssertionFailedError` (une sous-classe d'`AssertionError`), que JUnit transforme en verdict FAIL avec le message fourni.

4.7 AssertJ : pourquoi une deuxième bibliothèque d'assertions ?

JUnit ne propose pour les doubles qu'un delta **absolu** : `assertEquals(2.0, y, 1e-12)` vérifie $|y - 2| \leq 10^{-12}$. Or le cours exige une mesure **relative** (§ 2.8). AssertJ la fournit, avec une syntaxe « fluide » qui se lit comme une phrase :

```
assertThat(r[0] * r[1]).isCloseTo(c / a, withinPercentage(1e-10));
//  « affirme que r0·r1 est proche de c/a à 1e-10 % près »
```

`withinPercentage(p)` accepte |obtenu − attendu| ≤ p/100 × |attendu|. Attention à l'unité : c'est un **pourcentage**. Correspondances utilisées dans le projet :

Écriture	Erreur relative tolérée	Tests
`withinPercentage(1e-9)`	10^{-11}	construction, constructionGeneree
`withinPercentage(1e-10)`	10^{-12}	viete
`withinPercentage(1e-12)`	10^{-14}	differentiel

Autre avantage : les messages d'échec sont très explicites, par exemple pour Viète sur la version naïve :

```
Expecting actual:
  0.7450580596923828
to be close to:
  1.0
by less than 1.0E-10% but difference was 25.49419403076172%.
```

4.8 La bibliothèque standard Java mobilisée

API	Comportement utile
`Math.ulp(1.0)`	renvoie 2^{-52} : l'epsilon machine ε (constante `EPS`)
`Math.getExponent(m)`	l'exposant binaire e tel que m = 1,xxx × 2^e
`Math.scalb(x, k)`	x × 2^k, calculé exactement en modifiant l'exposant
`Math.copySign(x, s)`	|x| avec le signe de s
`Math.min` / `Math.max`	tri des deux racines ; renvoient NaN si un argument est NaN
`new BigDecimal(double)`	la valeur **exacte** du double (ex. 0.1 → 0.1000000000000000055511…), à la différence de `BigDecimal.valueOf` qui part de l'écriture décimale
`MathContext(50)`	précision de 50 chiffres significatifs pour `sqrt` et `divide`
`BigDecimal.signum()`	signe exact (−1, 0, 1) d'un calcul sans arrondi : signe exact de Δ
`new Random(563)`	générateur pseudo-aléatoire à **graine fixe** : toujours la même suite de nombres
`Arrays.stream`, `flatMap`, `IntStream.rangeClosed`	construire le produit cartésien et la suite des 20 tests

4.9 VS Code et l'Extension Pack for Java

VS Code est un éditeur ; c'est l'**Extension Pack for Java** (`vscjava.vscode-java-pack`) qui le transforme en environnement Java. Extensions installées sur votre machine :

Extension	Rôle
Language Support for Java (Red Hat)	serveur de langage : comprend le code, compile en continu, signale les erreurs, importe le projet Maven
Test Runner for Java	le **Test Explorer** (icône fiole) : découvre, lance et affiche les tests JUnit
Debugger for Java	points d'arrêt, exécution pas à pas, inspection des variables (Debug Test)
Maven for Java	vue Maven, exécution des phases (`test`, `clean`…) via le wrapper
Project Manager for Java, Gradle for Java	vue des dépendances ; support Gradle (non utilisé)

Les réglages partagés du projet (`.vscode/settings.json`) :

Clé	Effet
`"java.configuration.updateBuildConfiguration": "automatic"`	si `pom.xml` change, VS Code réimporte le projet sans poser de question

Clé	Effet
`"java.compile.nullAnalysis.mode": "disabled"`	désactive l'analyse des annotations de nullité (évite des avertissements inutiles)
`"maven.executable.preferMavenWrapper": true`	l'extension Maven utilise `mvnw.cmd` plutôt qu'un Maven installé
`"files.encoding": "utf8"`	les accents des fichiers sont lus et écrits correctement

Différence importante : VS Code n'utilise pas Surefire

Quand vous cliquez ▶ dans le Test Explorer, VS Code compile avec son propre compilateur et lance les tests directement via la JUnit Platform, **sans passer par Maven ni Surefire**. Les profils et le filtre de tags du `pom.xml` ne s'appliquent donc pas : c'est pour cela que vous pouvez lancer `SolveurNaifTest` directement depuis VS Code, alors que `mvnw test` l'exclut.

5. L'importance des annotations « @ »

> En cours, l'enseignant a insisté sur les « arobases ». Ce sont les **annotations** Java. Sans elles, JUnit ne trouverait **aucun** test : elles sont le langage par lequel on dit à JUnit « ceci est un test, voici ses données, voici son nom ».

5.1 Qu'est-ce qu'une annotation ?

Une annotation est une **métadonnée** attachée à un élément du code (classe, méthode, paramètre…). Elle s'écrit `@Nom` ou `@Nom(paramètres)` juste avant l'élément. Point essentiel : **une annotation ne fait rien par elle-même**. Elle ne modifie pas le comportement de la méthode ; elle est une **étiquette lisible par un outil** :

- par le **compilateur** : `@Override` lui demande de vérifier qu'on redéfinit bien une méthode ;
- par un **framework à l'exécution** : JUnit lit `@Test`, `@ParameterizedTest`… grâce à la **réflexion** Java (la capacité d'un programme à inspecter ses propres classes et méthodes pendant qu'il tourne). Pour cela, les annotations JUnit sont déclarées avec `@Retention(RUNTIME)` : elles sont conservées dans le `.class` et visibles à l'exécution.

```
@ParameterizedTest(name = "[{index}] ({0}, {1}, {2})")   // ← annotation avec un paramètre nommé
@CsvSource({"1, 1e8, 1", "3, 7, 2", "2, -4, 1.99"})      // ← annotation avec un tableau de chaînes
@DisplayName("Propriété : résidu (erreur inverse)")      // ← annotation avec une valeur
void residu(double a, double b, double c) { … }          // ← l'élément annoté : une méthode
```

5.2 Pourquoi JUnit dépend entièrement des annotations

Pour exécuter des tests, le moteur JUnit doit répondre à trois questions : **quelles** méthodes sont des tests ? **avec quelles données** ? **comment** les présenter ? Les annotations y répondent :

1. **Découverte** : le moteur Jupiter parcourt les classes de test et, par réflexion, retient les méthodes portant `@Test`, `@ParameterizedTest` ou `@TestFactory`. Une méthode sans annotation est une méthode ordinaire, **jamais exécutée comme test** (c'est le cas de `erreurRelative` et de `g`).
2. **Alimentation** : pour un test paramétré, `@CsvSource` ou `@MethodSource` indiquent d'où viennent les arguments.
3. **Présentation et filtrage** : `@DisplayName` et `name =` donnent des noms lisibles ; `@Tag` permet de choisir quels tests lancer.

> **Un peu d'histoire**
> Avant JUnit 4 (2006), un test devait hériter de la classe `TestCase` et son nom commencer par `test` : la découverte reposait sur des conventions de nommage. Depuis JUnit 4, et encore plus avec JUnit 5, ce sont les annotations qui portent cette information : le code est plus libre, plus lisible, et peut porter des données (tests paramétrés).

> **Piège : oublier l'annotation**
> Si l'on supprime `@ParameterizedTest` au-dessus de `residu`, le projet compile toujours, mais la méthode n'est plus un test : JUnit ne l'exécute plus et **aucune alerte** n'apparaît. Le nombre de tests passe silencieusement de 84 à 81. C'est pourquoi il faut toujours vérifier le **nombre** de tests exécutés (`Tests run: 84`), pas seulement l'absence de rouge.

5.3 Inventaire de toutes les annotations du projet

Annotation	Bibliothèque	Où	Rôle	Si on l'enlève
`@Test`	junit-jupiter-api	`aNul`	déclare un test simple, sans paramètre, exécuté une fois	la méthode n'est plus exécutée
`@ParameterizedTest`	junit-jupiter-params	9 méthodes	déclare un test exécuté **une fois par jeu d'arguments** ; `name` fixe le nom affiché de chaque cas	la méthode n'est plus exécutée
`@CsvSource`	junit-jupiter-params	8 méthodes	fournit les arguments sous forme de lignes CSV (une ligne = un cas)	erreur : test paramétré sans source
`@MethodSource`	junit-jupiter-params	`combinaisons`	fournit les arguments par une méthode `static` qui renvoie un `Stream<Arguments>`	erreur : test paramétré sans source
`@TestFactory`	junit-jupiter-api	`constructionGeneree`	déclare une **fabrique** de tests : la méthode renvoie des `DynamicTest` créés à l'exécution	les 20 tests disparaissent
`@DisplayName`	junit-jupiter-api	les 11 méthodes + 2 classes	nom lisible (accents, espaces, symboles) dans VS Code et les rapports	le nom technique de la méthode s'affiche
`@Tag("naif")`	junit-jupiter-api	`SolveurNaifTest`	étiquette tous les tests de la classe ; Surefire filtre sur ce tag	`mvnw test` lancerait aussi la version naïve → build rouge
`@Override`	java.lang (JDK)	`g` dans les 2 sous-classes	demande au compilateur de vérifier qu'on redéfinit bien `g`	le code marche, mais une faute de signature ne serait plus signalée comme telle

Comptage des tests à partir des annotations : 1 `@Test` (1 cas) + 9 `@ParameterizedTest` (8 + 3 + 3 + 2 + 3 + 2 + 3 + 36 + 3 = 63 cas) + 1 `@TestFactory` (20 tests dynamiques) = **84 tests**.

5.4 Zoom : @ParameterizedTest et @CsvSource

C'est l'annotation qui traduit directement le modèle du cours en code (diapo 69) : **chaque ligne CSV = un cas (DE, RA, Tol)** ; **la méthode = g + oracle** ; **ajouter un cas = ajouter une ligne**.

```
@ParameterizedTest(name = "[{index}] {6} : ({0}, {1}, {2}) → [{3}, {4}], Tol = {5}")
@CsvSource(textBlock = """
        # a,      b,      c,                RA r1,              RA r2,              Tol,   diagnostic
          1,      -3,     2,                1.0,                2.0,                2e-14, cas nominal
          1,      1e8,    1,                -99999999.99999999, -1e-8,              5e-15, élimination
catastrophique
        """)
void parReference(double a, double b, double c, double r1, double r2, double tol, String diagnostic)
```

- **Découpage** : chaque ligne est coupée sur les virgules ; la colonne i devient le paramètre i de la méthode.
- **Conversion implicite** : JUnit convertit chaque texte vers le type du paramètre : `"1e8"` → `double` 1.0E8, `"élimination catastrophique"` → `String`.
- **Commentaires** : dans un `textBlock`, une ligne commençant par `#` est ignorée (ici l'en-tête des colonnes).
- **Nom affiché** : dans `name`, `{index}` est le numéro du cas et `{0}` … `{6}` les arguments **tels qu'écrits dans le CSV** (avant conversion). Le cas 2 s'affiche (vérifié en exécutant la suite) : « [2] élimination catastrophique : (1, 1e8, 1) → [-99999999.99999999, -1e-8], Tol = 5e-15 ». Avec `@MethodSource`, les arguments sont déjà des `Double` : « [10] a = -2.0, b = 1.0E8, c = -4.0 ».

Piège rencontré pendant le développement

Une virgule dans un commentaire placé en fin de ligne CSV crée une colonne de trop. Les explications sur κ ont donc été déplacées en commentaires Java `//`, au-dessus du bloc.

5.5 Zoom : @MethodSource et les combinaisons de paramètres

C'est la réponse à la demande de l'enseignant : « chaque paramètre a son tableau de valeurs, et JUnit combine tous les cas possibles ». JUnit 5 n'a pas d'annotation « produit cartésien » toute faite : on écrit une petite méthode `static` qui le construit, et `@MethodSource` la désigne par son nom.

```
static final double[] VALEURS_A = {-2, 1, 3};          // 3 valeurs
static final double[] VALEURS_B = {-5, 0, 1, 1e8};     // 4 valeurs
static final double[] VALEURS_C = {-4, 0, 2};          // 3 valeurs

static Stream<Arguments> toutesLesCombinaisons() {     // 3 × 4 × 3 = 36 triplets
    return Arrays.stream(VALEURS_A).boxed().flatMap(a ->
            Arrays.stream(VALEURS_B).boxed().flatMap(b ->
                    Arrays.stream(VALEURS_C).mapToObj(c -> Arguments.of(a, b, c))));
}

@ParameterizedTest(name = "[{index}] a = {0}, b = {1}, c = {2}")
@MethodSource("toutesLesCombinaisons")
void combinaisons(double a, double b, double c) { … }
```

Les `flatMap` imbriqués jouent le rôle de trois boucles `for` imbriquées : pour chaque a, pour chaque b, pour chaque c, on produit `Arguments.of(a, b, c)`. Ajouter une valeur dans un tableau multiplie automatiquement le nombre de cas (ex. une 5e valeur de b → 45 cas).

5.6 Zoom : @TestFactory et les tests dynamiques

Avec `@Test` et `@ParameterizedTest`, les cas sont connus **à la compilation** (écrits dans le code). Avec `@TestFactory`, ils sont **fabriqués à l'exécution** : la méthode renvoie un flux de `DynamicTest`, chacun composé d'un nom et d'un bloc de code (lambda).

```
@TestFactory
Stream<DynamicTest> constructionGeneree() {
    Random alea = new Random(563);                      // graine fixe : reproductible
    return IntStream.rangeClosed(1, 20).mapToObj(i -> {
        … tirer r1 et r2 …
        return dynamicTest("cas " + i + " : racines choisies " + r1 + " et " + r2, () -> {
            double[] r = g(1, -(r1 + r2), r1 * r2);      // oracle par construction
            assertThat(r[0]).isCloseTo(Math.min(r1, r2), withinPercentage(1e-9));
            assertThat(r[1]).isCloseTo(Math.max(r1, r2), withinPercentage(1e-9));
        });
    });
}
```

Critère	@ParameterizedTest	@TestFactory (dynamique)
Cas connus	à l'écriture (CSV, méthode source)	à l'exécution (calculés, tirés au hasard…)
Signature	méthode avec paramètres, renvoie `void`	méthode sans paramètre, renvoie `Stream<DynamicTest>`
Cycle de vie	`@BeforeEach` / `@AfterEach` autour de chaque cas	autour de la fabrique entière, pas de chaque test dynamique
Usage typique	tableau de cas choisis à la main	génération de nombreux cas (aléatoires, issus d'un fichier, d'un calcul)

Si l'enseignant demande : « vos tests aléatoires ne sont-ils pas irreproductibles ? »

Non : la graine est **fixe** (`new Random(563)`). Le générateur produit toujours la même suite, donc les 20 cas sont toujours les mêmes et un échec se reproduit à l'identique (diapo 64 : « un FAIL se reproduit avant d'ouvrir un ticket »). Pour explorer d'autres cas, il suffit de changer la graine.

5.7 Zoom : @DisplayName, @Tag et @Override

- `@DisplayName` : sur une classe ou une méthode, remplace le nom technique par un texte libre. Dans VS Code on lit « Solveur NAÏF (formule du lycée) → Oracle par référence : N(y, RA) ≤ Tol → [2] élimination catastrophique… » au lieu de `SolveurNaifTest.parReference(double, double, …)[2]`. Les rapports XML de Surefire, eux, gardent le nom technique.
- `@Tag("naif")` : placé sur la classe, il s'applique à tous ses tests, y compris ceux hérités. Il ne change rien à l'exécution d'un test ; il sert uniquement au **filtrage** (Surefire `groups` / `excludedGroups`, ou les filtres du Test Explorer).
- `@Override` : annotation du JDK, lue par le compilateur seul. Elle garantit que `g` redéfinit bien la méthode abstraite de la suite. Ici, comme `g` est abstraite, une erreur de signature empêcherait de toute façon la compilation (la classe resterait abstraite) ; `@Override` rend le message d'erreur clair et documente l'intention.

5.8 Les annotations JUnit 5 non utilisées que l'enseignant peut citer

Annotation	Rôle	Pourquoi pas ici
`@BeforeEach` / `@AfterEach`	code exécuté avant / après chaque test (préparer / nettoyer le contexte)	le solveur est sans état : rien à préparer
`@BeforeAll` / `@AfterAll`	une fois avant / après tous les tests de la classe (méthode static)	idem
`@ValueSource`	source d'arguments à **un seul** paramètre (`ints`, `doubles`, `strings`…)	nos tests ont 2 à 7 paramètres
`@CsvFileSource`	comme `@CsvSource`, mais lit un fichier `.csv`	8 cas : plus lisible dans le code
`@EnumSource`, `@ArgumentsSource`	arguments depuis une énumération / une classe fournisseur	non nécessaire
`@Disabled`	désactive un test (il apparaît « ignoré »)	tous les tests sont actifs
`@Nested`	classe de test interne pour regrouper des tests	possible, mais l'héritage suffit
`@RepeatedTest(n)`	répète un test n fois	le code est déterministe
`@Timeout`	échec si le test dépasse une durée	calculs instantanés
`@TestInstance(PER_CLASS)`	une seule instance pour toute la classe	on garde l'isolation par défaut
`@ExtendWith`	branche une extension (ex. Mockito)	pas de doublure dans ce TP

À retenir

Les « @ » sont des **métadonnées** : elles ne calculent rien, elles **déclarent**. JUnit les lit par réflexion pour savoir **quoi** exécuter (`@Test`, `@ParameterizedTest`, `@TestFactory`), **avec quelles données** (`@CsvSource`, `@MethodSource`) et **comment le présenter ou le filtrer** (`@DisplayName`, `@Tag`). Sans annotation, pas de test.

6. Le code de production expliqué ligne par ligne

> Le code de production (`src/main`) est volontairement court : c'est le **système sous test**. Les deux algorithmes sont recopiés fidèlement des diapos 51 et 54, pour que les résultats soient ceux du cours.

6.1 Solveur.java : la structure de la classe

```
public final class Solveur {

    private Solveur() {
    }

    /** Système sous test g utilisé par défaut : la version stable. */
    public static double[] racines(double a, double b, double c) {
        return stable(a, b, c);
    }

    /** Système sous test g, version naïve (pour montrer ce que les oracles détectent). */
    public static double[] racinesNaif(double a, double b, double c) {
        return naif(a, b, c);
    }
    …
}
```

- `final` + **constructeur** `private` : c'est une **classe utilitaire**, comme `java.lang.Math`. On n'en crée jamais d'objet et on ne l'étend pas ; on appelle ses méthodes `static` directement : `Solveur.racines(1, -3, 2)`.
- `racines` est le point d'entrée « officiel » : c'est le prototype imposé par le TP (`Solveur.racines(a, b, c)`, diapo 50). Il délègue à la version **stable**.
- `racinesNaif` expose la version naïve avec **le même prototype**, uniquement pour que les tests puissent la juger avec les mêmes oracles.
- **Type de retour** `double[]` : un tableau de taille variable (0, 1 ou 2) représente naturellement les trois cas du discriminant. C'est la « liste de double » dont parlait l'enseignant.

6.2 La version naïve : naif(a, b, c)

```
static double[] naif(double a, double b, double c) {
    if (a == 0) throw new IllegalArgumentException("a = 0");   // (1) contrat : pas du 2nd degré
    double d = b * b - 4 * a * c;                              // (2) discriminant Δ
    if (d < 0) return new double[0];                           // (3) aucune racine réelle
    if (d == 0) return new double[]{-b / (2 * a)};             // (4) racine double
    double r1 = (-b - Math.sqrt(d)) / (2 * a);                 // (5) formule du lycée, signe −
    double r2 = (-b + Math.sqrt(d)) / (2 * a);                 // (6) formule du lycée, signe +
    return new double[]{Math.min(r1, r2), Math.max(r1, r2)};   // (7) tri croissant
}
```

Ligne	Ce qu'elle fait	Faiblesse numérique
(1)	refuse a = 0 en levant une exception (contrat)	aucune
(2)	calcule Δ = $b^2 − 4ac$ en double	**b^2 peut déborder** (+∞) ; ∞ − ∞ = **NaN** ; 4ac peut être absorbé
(3)	Δ < 0 → tableau vide	si Δ = NaN, le test est faux : le code continue avec NaN
(4)	Δ = 0 → une racine	idem avec NaN
(5)-(6)	les deux racines par (−b ± √Δ) / 2a	l'une des deux soustrait des nombres proches : **élimination catastrophique**
(7)	trie les racines (le signe de a peut inverser l'ordre)	`Math.min(NaN, NaN)` = NaN : le NaN passe tel quel

Si l'enseignant demande : « ce code est-il faux ? »

Il est **mathématiquement correct** (c'est la question de la diapo 51) mais **numériquement instable** : sur des entrées légales, il renvoie des résultats faux (25 % d'erreur, ±∞, NaN, 0) sans lever d'exception. C'est un défaut **silencieux**, le plus dangereux, que seuls des tests avec un oracle exigeant détectent.

6.3 La version stable : stable(a, b, c)

```
static double[] stable(double a, double b, double c) {
    if (a == 0) throw new IllegalArgumentException("a = 0");                  // (1) même contrat
    double m = Math.max(Math.abs(a), Math.max(Math.abs(b), Math.abs(c)));     // (2) plus grand |coefficient|
    int e = Math.getExponent(m);                                              // (3) son exposant binaire
    a = Math.scalb(a, -e);                                                    // (4) a · 2^-e  (exact)
    b = Math.scalb(b, -e);                                                    //     b · 2^-e
    c = Math.scalb(c, -e);                                                    //     c · 2^-e
    double d = b * b - 4 * a * c;                                             // (5) Δ sans débordement
    if (d < 0) return new double[0];                                          // (6)
    if (d == 0) return new double[]{-b / (2 * a)};                            // (7)
    double q = -0.5 * (b + Math.copySign(Math.sqrt(d), b));                   // (8) pas de soustraction
    double r1 = q / a, r2 = c / q;                                            // (9) Viète : r1 · r2 = c / a
    return new double[]{Math.min(r1, r2), Math.max(r1, r2)};                  // (10) tri
}
```

Ligne	Ce qu'elle fait et pourquoi
(1)	même contrat que la version naïve : les deux versions sont interchangeables pour les tests
(2)-(3)	trouve l'ordre de grandeur binaire du plus grand coefficient (ex. 10^{200} → e = 664)
(4)	divise les trois coefficients par 2e : opération exacte, équation équivalente, plus grand coefficient ramené dans [1, 2[
(5)	Δ ne peut plus déborder ni produire ∞ − ∞
(6)-(7)	mêmes cas particuliers que la version naïve
(8)	`copySign` donne à √Δ le signe de b : b + signe(b)·√Δ additionne deux nombres de même signe
(9)	r1 = q/a est la racine « sûre » ; r2 = c/q vient de Viète : une division, sans soustraction
(10)	tri croissant pour respecter le contrat

Pour aller plus loin : modifier les paramètres a, b, c dans la méthode

Les lignes (4) réaffectent les paramètres. En Java, les paramètres sont passés **par valeur** : modifier `a` dans `stable` ne change pas la variable de l'appelant. C'est donc sans effet de bord.

6.4 Comparaison synthétique des deux versions

Situation	naif	stable
Cas nominal (1, −3, 2)	[1.0, 2.0]	[1.0, 2.0]
$b^2 \gg 4ac$ (1, 10^8, 1)	petite racine fausse de 25 %	[−1.0E8, −1.0E-8]
b très grand (1, 10^{200}, 1)	[−∞, +∞]	[−1.0E200, −1.0E-200]
coefficients énormes (10^{200}, …)	[NaN, NaN]	[1.0, 2.0]
a et c minuscules (10^{-200}, 1, 10^{-200})	petite racine = 0.0	[−1.0E200, −1.0E-200]
Δ ≈ 0 (mal conditionné)	juste à √ε près	identique
Coût	2 racines carrées	1 racine carrée + 3 `scalb` (négligeable)

6.5 Demo.java : afficher n'est pas tester

```
public static void main(String[] args) {
    Object[][] cas = {                                  // a, b, c, diagnostic, résultat attendu
            {1, -3, 2, "cas nominal", "[1.0, 2.0]"},
            {1, 1e8, 1, "elimination catastrophique", "[-1.0E8, -1.0E-8]"},
            …
    };
    String format = "%-28s | %-34s | %-42s | %-42s | %s%n";
    System.out.printf(format, "Diagnostic", "(a, b, c)", "Naif", "Stable", "Attendu");
    for (Object[] l : cas) {
        double a = ((Number) l[0]).doubleValue(), …;      // Integer ou Double → double
        System.out.printf(format, l[3], "(" + a + ", " + b + ", " + c + ")",
                Arrays.toString(Solveur.racinesNaif(a, b, c)), Arrays.toString(Solveur.racines(a, b, c)), l[4]);
    }
}
```

- `Object[][]` mélange des nombres et des textes ; `((Number) l[0]).doubleValue()` convertit un `Integer` (1) ou un `Double` (1e8) en `double`.
- `printf` avec `%-28s` aligne les colonnes ; la sortie est en ASCII (sans accents) pour rester lisible dans la console Windows.

Sortie réelle (colonnes raccourcies) :

```
Diagnostic                  | (a, b, c)                       | Naif                              | Stable
cas nominal                 | (1.0, -3.0, 2.0)                | [1.0, 2.0]                        | [1.0, 2.0]
elimination catastrophique  | (1.0, 1.0E8, 1.0)               | [-1.0E8, -7.450580596923828E-9]   | [-1.0E8,
-1.0E-8]
b^2 deborde en +Inf         | (1.0, 1.0E200, 1.0)             | [-Infinity, Infinity]             | [-1.0E200,
-1.0E-200]
Inf - Inf = NaN             | (1.0E200, -3.0E200, 2.0E200)    | [NaN, NaN]                        | [1.0, 2.0]
absorption                  | (1.0E-200, 1.0, 1.0E-200)       | [-1.0E200, 0.0]                   | [-1.0E200,
-1.0E-200]
Delta ~ 0 (mal conditionne) | (1.0, -2.0, 0.9999999999999999) | [0.9999999894632878, 1.00000001…] |
[0.9999999894632878, 1.00000001…]
```

> **À retenir : Demo vs tests**
>
> `Demo` **montre** les résultats à un humain, qui doit les lire et juger lui-même (oracle humain, coûteux et non automatisable). Les tests **décident** automatiquement, à chaque exécution, avec un critère écrit (N ≤ Tol). C'est la différence que l'enseignant a soulignée : « afficher des résultats, ce n'est pas tester ». `Demo` n'est d'ailleurs pas exécutée par `mvnw test` : elle n'a aucune annotation de test et n'est pas dans `src/test`.

7. La suite de tests SuiteOraclesSolveur en détail

> C'est le fichier le plus important du projet. Il contient les 11 méthodes de test, organisées en blocs qui suivent la progression du chapitre III : d'abord l'oracle par référence, puis le contrat, puis les oracles sans résultat attendu, enfin la génération de cas.

7.1 Plan du fichier

Bloc	Méthode	Annotations	Cas	Famille d'oracle	Critère de décision (oracle)
1	`parReference`	@ParameterizedTest @CsvSource	8	référence	nombre de racines, puis N = |y − RA|/|RA| ≤ Tol
2	`aNul`	@Test	1	contrat	exception `IllegalArgumentException` levée
2	`discriminantNegatif`	@ParameterizedTest @CsvSource	3	contrat	tableau vide
3	`residu`	@ParameterizedTest @CsvSource	3	propriété	$\|ar^2 + br + c\| \le 8\varepsilon(\|a\|r^2 + \|br\| + \|c\|)$
3	`viete`	@ParameterizedTest @CsvSource	2	propriété	somme ≈ −b/a et produit ≈ c/a à 10^{-12}
3	`echelle`	@ParameterizedTest @CsvSource	3	métamorphique	racines(1024a, 1024b, 1024c) = racines(a, b, c) strictement
3	`oppose`	@ParameterizedTest @CsvSource	2	métamorphique	racines(a, −b, c) = −racines(a, b, c) strictement
3	`construction`	@ParameterizedTest @CsvSource	3	construction	retrouver r_1, r_2 choisis à 10^{-11}
4	`combinaisons`	@ParameterizedTest @MethodSource	36	propriété	nombre de racines = signe exact de Δ + 1, puis résidu
5	`constructionGeneree`	@TestFactory	20	construction	retrouver r_1, r_2 générés à 10^{-11}
6	`differentiel`	@ParameterizedTest @CsvSource	3	différentiel	grande racine ≈ calcul BigDecimal 50 chiffres à 10^{-14}
Total			**84**		

7.2 Les éléments communs

```java
abstract class SuiteOraclesSolveur {

    /** ε = ulp(1) = 2^-52 ≈ 2,2·10^-16 en double. */
    static final double EPS = Math.ulp(1.0);

    /** Le système sous test g : renvoie les racines triées de a·x² + b·x + c = 0. */
    abstract double[] g(double a, double b, double c);

    /** Mesure d'erreur N : erreur relative |y − RA| / |RA| (aucun RA n'est nul dans nos cas). */
    static double erreurRelative(double y, double ra) {
        return Math.abs(y - ra) / Math.abs(ra);
    }
```

- `abstract class` : la classe ne peut pas être instanciée ; elle sert de modèle aux classes concrètes.
- `EPS` : ε, l'unité de toutes les tolérances. `static final` = constante partagée.
- `g` : le système sous test du modèle du cours, sans corps. Tous les tests appellent `g`, jamais `Solveur` directement.
- `erreurRelative` : la mesure N, écrite une fois et réutilisée (pas d'annotation : ce n'est pas un test).

7.3 Bloc 1 : l'oracle par référence (parReference)

```java
@ParameterizedTest(name = "[{index}] {6} : ({0}, {1}, {2}) → [{3}, {4}], Tol = {5}")
@CsvSource(textBlock = """ … 8 lignes … """)
@DisplayName("Oracle par référence : N(y, RA) ≤ Tol")
void parReference(double a, double b, double c, double r1, double r2, double tol, String diagnostic) {
    double[] y = g(a, b, c);                        // y = g(DE)
    double[] ra = (r1 == r2) ? new double[]{r1} : new double[]{r1, r2};
    assertEquals(ra.length, y.length, "nombre de racines");
    for (int i = 0; i < ra.length; i++) {
        double n = erreurRelative(y[i], ra[i]);     // N
        assertTrue(n <= tol,                        // verdict : N ≤ Tol (faux si y = NaN)
                "racine " + i + " : y = " + y[i] + ", RA = " + ra[i] + ", N = " + n + " > Tol = " + tol);
    }
}
```

Lecture ligne par ligne :

1. `y = g(a, b, c)` : exécution du système sous test sur les données d'entrée (DE).
2. Construction du RA : convention du CSV, si r1 == r2 on attend **une seule** racine (racine double).
3. **Première assertion** : le nombre de racines. Si y n'a pas la bonne taille, inutile de comparer les valeurs (et on évite une `ArrayIndexOutOfBoundsException` qui donnerait ERROR au lieu de FAIL).
4. Pour chaque racine : calcul de N, puis **verdict** `assertTrue(n <= tol, message)`.
5. Le **message** d'échec contient tout ce qu'il faut pour diagnostiquer sans débogueur : y, RA, N et Tol. Exemple réel avec la version naïve : `racine 1 : y = -7.450580596923828E-9, RA = -1.0E-8, N = 0.2549419403076172 > Tol = 5.0E-15`.

Les 8 cas et leur raison d'être

#	DE (a, b, c)	RA	Tol	Pourquoi ce cas
1	(1, −3, 2)	[1, 2]	2e−14	cas nominal : vérifie que le test lui-même fonctionne
2	(1, 1e8, 1)	[−99999999.99999999, −1e−8]	5e−15	élimination catastrophique ($b^2 \gg 4ac$)
3	(1, 1e200, 1)	[−1e200, −1e−200]	5e−15	débordement de b^2 dans un calcul intermédiaire
4	(1e200, −3e200, 2e200)	[1, 2]	2e−14	$\infty - \infty$ = NaN (même équation que le cas 1, multipliée par 10^{200})
5	(1e−200, 1, 1e−200)	[−1e200, −1e−200]	5e−15	absorption / dépassement inférieur de 4ac
6	(−2, 3, 5)	[−1, 2.5]	4e−15	a négatif : vérifie le tri (le signe de a inverse l'ordre de la formule)
7	(1, −2, 1)	[1]	0	racine double : Δ = 0 exactement, une seule racine attendue
8	(1, −2, 0.9999999999999999)	[0.99999998946…, 1.00000001053…]	5e−7	bonus Δ ≈ 0 : problème mal conditionné, Tol justifiée par κ ≈ $1{,}9 \cdot 10^8$

> **D'où viennent les RA ?**
> Ils ont été calculés **à part**, avec le module `Decimal` de Python à 1 200 chiffres, à partir des valeurs **exactes** des doubles d'entrée (par exemple 0.9999999999999999 vaut exactement $1 - 2^{-53}$). Un RA ne doit jamais être calculé avec le code testé lui-même, sinon le test vérifierait le programme contre lui-même. Remarque : écrit dans le CSV, −99999999.99999999 est arrondi par Java au double le plus proche (−9.999999999999999E7) ; la version stable renvoie −1.0E8, à 1 ulp près, soit N ≈ $1{,}5 \cdot 10^{-16}$, bien sous Tol.

7.4 Bloc 2 : le contrat (aNul, discriminantNegatif)

```
@Test
@DisplayName("a = 0 : IllegalArgumentException attendue")
void aNul() {
    assertThrows(IllegalArgumentException.class, () -> g(0, 2, 1));
}

@ParameterizedTest(name = "[{index}] ({0}, {1}, {2}) : Δ < 0 → aucune racine réelle")
@CsvSource({"1, 0, 1", "1, 1, 1", "2, -4, 2.01"})
@DisplayName("Δ < 0 : tableau vide")
void discriminantNegatif(double a, double b, double c) {
    assertEquals(0, g(a, b, c).length);
}
```

- `aNul` illustre le verdict « PASS sur exception attendue » (diapo 63) : `assertThrows` exécute la lambda `() -> g(0, 2, 1)` et réussit seulement si elle lève `IllegalArgumentException`. Si aucune exception n'est levée, ou une autre, le test échoue. La lambda est nécessaire pour **retarder** l'appel : `assertThrows` doit pouvoir l'exécuter dans un bloc `try/catch`.
- `discriminantNegatif` : Δ vaut −4, −3 et 16 − 16,08 = −0,08. Le troisième cas est proche de 0 : il vérifie que l'arrondi ne fait pas basculer le signe.

7.5 Bloc 3 : les oracles sans résultat attendu

C'est la réponse au « problème de l'oracle » : on ne connaît pas toujours RA, mais on connaît des **propriétés** que toute bonne réponse doit respecter.

a) Le résidu (erreur inverse)

```
@CsvSource({"1, 1e8, 1", "3, 7, 2", "2, -4, 1.99"})
void residu(double a, double b, double c) {
    for (double r : g(a, b, c)) {
        double res = Math.abs(a * r * r + b * r + c);                  // à quel point r annule le polynôme
        double ech = Math.abs(a) * r * r + Math.abs(b * r) + Math.abs(c); // échelle des termes
        assertTrue(res <= 8 * EPS * ech, "r = " + r + " : résidu " + res + " > " + 8 * EPS * ech);
    }
}
```

Une vraie racine annule le polynôme. Mais en double, le résidu n'est jamais exactement nul : il faut le comparer à **l'échelle des termes** (diapo 78), sinon un résidu de 10^{-9} dû aux arrondis passerait pour un bug sur de grands coefficients. Le rapport res/ech est l'**erreur inverse** : la plus petite perturbation relative des coefficients pour laquelle r serait une racine exacte. Un bon algorithme donne quelques ε ; le facteur 8 couvre les arrondis de l'algorithme et du calcul du résidu lui-même.
Version naïve sur (1, 10^8, 1) : r = $-7{,}45\cdot10^{-9}$ donne res = |−0,745 + 1| = 0,2549 contre une borne de $3{,}1\cdot10^{-15}$: FAIL.

b) Les relations de Viète

```
@CsvSource({"1, 1e8, 1", "3, 7, 2"})
void viete(double a, double b, double c) {
    double[] r = g(a, b, c);
    assertThat(r[0] + r[1]).isCloseTo(-b / a, withinPercentage(1e-10));
    assertThat(r[0] * r[1]).isCloseTo(c / a, withinPercentage(1e-10));
}
```

Somme et produit des racines doivent valoir −b/a et c/a. Point subtil : sur (1, 10^8, 1), la **somme** de la version naïve est juste ($-10^8 - 7{,}45\cdot10^{-9} \approx -10^8$: la grande racine domine) ; c'est le **produit** (0,745 au lieu de 1) qui révèle l'erreur sur la petite racine. D'où l'intérêt de tester les deux relations.

c) Relation métamorphique : le changement d'échelle

```
@CsvSource({"3, 7.1, 2", "1, 1e8, 1", "-2, 3, 5"})
void echelle(double a, double b, double c) {
    double k = 1024;                              // 2^10 : produit exact
    assertArrayEquals(g(a, b, c), g(k * a, k * b, k * c)); // strict
}
```

Transformation T : multiplier les trois coefficients par k. Relation R : mêmes racines. On ne connaît pas les racines, mais on sait comment elles doivent (ne pas) varier. Avec **$k = 2^{10}$**, la multiplication ne change que l'exposant : aucun arrondi, donc on peut exiger l'**égalité stricte**. Avec k = 10, ce serait faux : on a vérifié que racines(3, 7.1, 2) = [−2.039844343835306, −0.32682232283136026] alors que racines(30, 71, 20) = [−2.0398443438353064, −0.3268223228313602] (dernier chiffre différent) ; il faudrait une tolérance.

d) Relation métamorphique : l'opposé

```
@CsvSource({"3, 7.1, 2", "1, 1e8, 1"})
void oppose(double a, double b, double c) {
    double[] r = g(a, b, c);
    double[] s = g(a, -b, c);
    assertEquals(-r[0], s[1], 0.0);               // les racines sont triées :
    assertEquals(-r[1], s[0], 0.0);               // l'opposé inverse l'ordre
}
```

Si x est racine de $ax^2 + bx + c$, alors −x est racine de $ax^2 − bx + c$ (remplacer x par −x). Comme les racines sont triées, la plus petite de l'une correspond à l'opposé de la plus grande de l'autre. Delta 0.0 : égalité stricte, car changer le signe de b est exact et les calculs sont symétriques.

> **Pourquoi ces deux relations ne détectent pas la version naïve**
>
> La version naïve est **fausse mais cohérente** : sur (1024, $1024 \cdot 10^8$, 1024), Δ est multiplié par 2^{20} et √Δ par 2^{10}, exactement ; elle refait donc **la même erreur** : [−1.0E8, −7.450580596923828E-9] dans les deux cas. De même, racinesNaif(1, $−10^8$, 1) = [7.45E-9, 1.0E8], l'opposé exact du résultat faux. La relation est respectée, le test est vert. C'est la leçon de la diapo 77.

e) L'oracle par construction

```
@CsvSource({"-1e8, -1e-8", "1, 2", "-3, 1e-5"})
void construction(double r1, double r2) {
    double[] r = g(1, -(r1 + r2), r1 * r2);       // (x − r1)(x − r2) = x² − (r1 + r2)x + r1·r2
    assertThat(r[0]).isCloseTo(Math.min(r1, r2), withinPercentage(1e-9));
    assertThat(r[1]).isCloseTo(Math.max(r1, r2), withinPercentage(1e-9));
}
```

On inverse le problème : on **choisit la réponse**, puis on fabrique l'équation correspondante. Le RA devient connu par construction. Limite (diapo 80) : $b = −(r_1 + r_2)$ et $c = r_1 \cdot r_2$ sont eux-mêmes arrondis, donc l'équation construite n'a pas exactement r_1 et r_2 pour racines ; d'où une tolérance de 10^{-11} (et non ε). Version naïve sur $(−10^8, −10^{-8})$: $−1{,}49 \cdot 10^{-8}$ au lieu de $−10^{-8}$, soit 49 % d'erreur : FAIL.

7.6 Bloc 4 : toutes les combinaisons de paramètres

```java
@ParameterizedTest(name = "[{index}] a = {0}, b = {1}, c = {2}")
@MethodSource("toutesLesCombinaisons")
void combinaisons(double a, double b, double c) {
    BigDecimal A = new BigDecimal(a), B = new BigDecimal(b), C = new BigDecimal(c);
    int signeDelta = B.multiply(B).subtract(A.multiply(C).multiply(BigDecimal.valueOf(4))).signum();
    double[] r = g(a, b, c);
    assertEquals(signeDelta + 1, r.length, "nombre de racines");   // Δ<0 → 0, Δ=0 → 1, Δ>0 → 2
    for (double x : r) { … même vérification du résidu que residu() … }
}
```

Ici aucun RA n'est écrit : 36 triplets sont produits automatiquement (§ 5.5). L'oracle combine deux vérifications :

1. **Le nombre de racines**, déduit du **signe exact** de Δ. On ne peut pas utiliser Δ calculé en double : près de 0, l'arrondi pourrait se tromper de signe, et l'oracle serait aussi faux que le programme. `new BigDecimal(double)` reprend la valeur exacte de chaque coefficient, et `multiply` / `subtract` sans `MathContext` calculent **sans aucun arrondi**. Astuce : `signum()` vaut −1, 0 ou 1, donc `signum() + 1` = nombre de racines attendu.
2. **Le résidu** de chaque racine (erreur inverse ≤ 8ε).

Répartition des 36 combinaisons	Nombre	Exemples
Δ < 0 → 0 racine	7	(−2, −5, −4), (1, 0, 2), (3, 1, 2)
Δ = 0 → 1 racine (x = 0)	3	(−2, 0, 0), (1, 0, 0), (3, 0, 0)
Δ > 0 → 2 racines	26	(1, −5, 2), (3, 1e8, −4)…
FAIL de la version naïve	6	n° 10, 12, 22, 24, 34, 36 : **exactement** les cas b = 10^8 et c ≠ 0

Pourquoi b = 10^8 avec c = 0 ne fait pas échouer la version naïve : la petite racine vaut alors exactement 0, et $-b + \sqrt{(b^2)} = 0$ est calculé sans erreur. L'élimination n'apparaît que si la petite racine est non nulle.

7.7 Bloc 5 : les 20 tests générés dynamiquement

```java
@TestFactory
@DisplayName("Tests dynamiques : 20 équations générées par construction")
Stream<DynamicTest> constructionGeneree() {
    Random alea = new Random(563);                  // graine fixe
    return IntStream.rangeClosed(1, 20).mapToObj(i -> {
        int e1 = alea.nextInt(-8, 9);               // exposant de r1 dans [-8, 8]
        int e2 = e1 + 1 + alea.nextInt(0, 8);       // e2 > e1 : racines bien séparées
        double r1 = (alea.nextBoolean() ? 1 : -1) * (1 + alea.nextInt(9)) * Math.pow(10, e1);
        double r2 = (alea.nextBoolean() ? 1 : -1) * (1 + alea.nextInt(9)) * Math.pow(10, e2);
        return dynamicTest("cas " + i + " : racines choisies " + r1 + " et " + r2, () -> { … });
    });
}
```

Règles de génération : chaque racine vaut ± m × 10^e avec m entre 1 et 9 ; l'exposant de r_2 dépasse celui de r_1 de 1 à 8. Les racines ont donc des ordres de grandeur différents : elles sont bien séparées, leur conditionnement est petit, et la tolérance de 10^{-11} est largement justifiée. Les 20 cas réels (graine 563) :

Cas	r_1	r_2	Naïf	Cas	r_1	r_2	Naïf
1	9.0E-7	-0.004	PASS	11	-2.0E-7	0.003	PASS
2	4.0	40.0	PASS	12	-0.03	-3000000.0	FAIL
3	-5000000.0	5.0E11	PASS	13	-10.0	2000000.0	PASS
4	9.0E-5	-6.000000000000001E-4	PASS	14	1.0	-1.0E7	PASS
5	0.006	0.9	PASS	15	-2.0E-7	0.006	PASS
6	9000000.0	8.0E12	PASS	16	-4.9999999999999996E-6	0.08	PASS
7	-40.0	-60000.0	PASS	17	-5.0E-8	-1.0	FAIL

Cas	r_1	r_2	Naïf	Cas	r_1	r_2	Naïf
8	-600000.0	-6000000.0	PASS	18	0.2	-60.0	PASS
9	6.0000000000000001E-8	0.30000000000000004	FAIL	19	-2.0E-7	30.0	FAIL
10	200000.0	-9.0E8	PASS	20	-400000.0	-3.0E8	PASS

Les 4 échecs de la version naïve ont tous un rapport entre les racines d'au moins $5{\cdot}10^6$: la petite racine subit l'élimination catastrophique (ex. cas 9 : $5{,}999999999617422{\cdot}10^{-8}$ au lieu de $6{\cdot}10^{-8}$). Un grand rapport ne garantit pas l'échec (cas 14, rapport 10^7, passe : l'arrondi tombe juste) : c'est pourquoi on multiplie les cas. Les valeurs comme `6.000000000000001E-8` viennent de `Math.pow(10, e)`, lui-même arrondi.

7.8 Bloc 6 : l'oracle différentiel en BigDecimal

```java
@CsvSource({"1, 1e8, 1", "3, 7.1, 2", "1e-3, -5, 1e3"})
void differentiel(double a, double b, double c) {   // a > 0 : (−b + √Δ)/2a est la plus grande racine
    MathContext mc = new MathContext(50);
    BigDecimal A = new BigDecimal(a), B = new BigDecimal(b);
    BigDecimal C = new BigDecimal(c), DEUX = BigDecimal.valueOf(2);
    BigDecimal s = B.multiply(B).subtract(A.multiply(C).multiply(BigDecimal.valueOf(4))).sqrt(mc);
    BigDecimal ref = B.negate().add(s).divide(A.multiply(DEUX), mc);
    assertThat(g(a, b, c)[1]).isCloseTo(ref.doubleValue(), withinPercentage(1e-12));
}
```

On compare le programme à une **autre implémentation** : la formule du lycée, mais calculée en `BigDecimal` à **50 chiffres significatifs**. Même si l'élimination fait perdre environ 16 chiffres, il en reste environ 34, bien plus que les 16 d'un double : la formule naïve redevient une référence fiable (diapo 81). Comme a > 0 dans les trois cas, (−b + √Δ)/2a est la **plus grande** racine, d'où la comparaison avec `g(…)[1]`.

DE	Référence (50 chiffres)	Stable	Naïf
(1, 1e8, 1)	−1.00000000000000010000000000000002E−8	-1.0E-8	-7.450580596923828E-9
(3, 7.1, 2)	−0.32682322283136023426272347 4169…	-0.32682322283136026	-0.32682322283136026
(1e−3, −5, 1e3)	4791.28784747791989901273712 18…	4791.287847477919	4791.287847477919

7.9 Récapitulatif : chaque élément du modèle, test par test

Test	DE	RA	N	Tol
parReference	CSV (8 cas)	CSV (Python 1 200 chiffres)	erreur relative	$c{\cdot}\kappa{\cdot}\varepsilon$ par cas
aNul	(0, 2, 1)	exception	—	—
discriminantNegatif	CSV (3 cas)	tableau vide	taille	exacte
residu	CSV (3 cas)	aucun	résidu / échelle	8ε
viete	CSV (2 cas)	aucun (−b/a, c/a)	erreur relative	10^{-12}
echelle, oppose	CSV + transformée	aucun (autre exécution)	égalité	0
construction	racines choisies	par construction	erreur relative	10^{-11}
combinaisons	produit cartésien	aucun (signe exact de Δ)	taille + résidu	8ε
constructionGeneree	générées (graine 563)	par construction	erreur relative	10^{-11}
differentiel	CSV (3 cas)	BigDecimal 50 chiffres	erreur relative	10^{-14}

8. SolveurNaifTest et SolveurStableTest : la différence

> Ces deux classes sont presque identiques, et c'est voulu : elles exécutent **exactement les mêmes 84 tests**, seul le **système sous test** change. La différence de résultat (84 verts contre 18 rouges) mesure donc uniquement la différence de qualité entre les deux algorithmes.

8.1 Les deux classes côte à côte

```java
package inf563.solveur;

import org.junit.jupiter.api.DisplayName;

/** Toute la suite contre la version
 *  stable : tout doit être VERT. */
@DisplayName("Solveur STABLE (…)")
class SolveurStableTest
        extends SuiteOraclesSolveur {

    @Override
    double[] g(double a, double b, double c) {
        return Solveur.racines(a, b, c);
    }
}
```

```java
package inf563.solveur;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Tag;

/** Toute la suite contre la version naïve :
 *  certains tests DOIVENT échouer. */
@Tag("naif")
@DisplayName("Solveur NAÏF (…)")
class SolveurNaifTest
        extends SuiteOraclesSolveur {

    @Override
    double[] g(double a, double b, double c) {
        return Solveur.racinesNaif(a, b, c);
    }
}
```

Mise en forme resserrée pour tenir sur deux colonnes (noms affichés abrégés en « … ») ; le code exact est en annexes D et E.

8.2 Ce qu'elles ont en commun

- Elles **héritent** (`extends`) de `SuiteOraclesSolveur` : elles reçoivent les 11 méthodes de test, les données, les oracles et les tolérances, sans en écrire une seule ligne.
- Elles **redéfinissent** (`@Override`) la seule méthode abstraite, `g`.
- Elles sont dans le même paquetage, sans modificateur `public`, et suivent le motif de nom `*Test`.
- Chacune produit **84 tests**.

8.3 Ce qui les différencie

Critère	SolveurStableTest	SolveurNaifTest
Système sous test g	`Solveur.racines` → `stable`	`Solveur.racinesNaif` → `naif`
Algorithme testé	formule stable + mise à l'échelle (diapo 54)	formule du lycée (diapo 51)
Annotation `@Tag`	aucune	`@Tag("naif")`
Nom affiché	« Solveur STABLE (formule stable + mise à l'échelle) »	« Solveur NAÏF (formule du lycée) »
Exécutée par `mvnw test`	**oui** (profil `defaut`)	**non** (tag exclu) ; avec `mvnw test -Pnaif` uniquement
Exécutable dans VS Code	oui	oui (VS Code ignore les profils Maven)
Résultat attendu	84 PASS → BUILD SUCCESS	66 PASS, 18 FAIL → BUILD FAILURE (voulu)
Rôle	**valider** l'implémentation livrée ; c'est le test de non-régression du projet	**évaluer les tests eux-mêmes** : prouver que les oracles détectent un défaut réel

8.4 Pourquoi garder une classe de test qui échoue ?

Parce qu'un test qui passe ne prouve rien tant qu'on ne sait pas s'il **peut** échouer. Si nos 84 tests étaient trop laxistes (tolérance trop large, mesure absolue, cas trop faciles), ils seraient verts aussi contre la version naïve. La classe `SolveurNaifTest` sert de **témoin** :

- elle répond à la consigne du TP : « exécuter toute la suite contre les deux versions et remplir le tableau détecte / ne détecte pas » ;
- elle montre **quels** oracles détectent le défaut et lesquels ne le voient pas (métamorphiques) ;
- elle annonce une idée du chapitre V (test de mutation) : la version naïve joue le rôle d'un **« mutant » réaliste** ; une bonne suite de tests doit le « tuer ».

On l'étiquette `@Tag("naif")` pour qu'elle ne casse pas le build normal : `mvnw test` reste vert (ce qu'exige une intégration continue), et l'on peut la lancer à la demande.

> **Si l'enseignant demande : « vos tests sont rouges, votre projet ne marche pas ? »**
> Le projet marche : le produit livré est la version **stable**, et ses 84 tests sont verts (`mvnw test` → `BUILD SUCCESS`). Les 18 rouges concernent la version **naïve**, gardée volontairement comme témoin : ils **prouvent** que la suite détecte l'élimination catastrophique, le débordement, le NaN et l'absorption. Un rouge attendu est une réussite du test.

8.5 Les 18 échecs de la version naïve, un par un

#	Test [cas]	DE	Ce que l'oracle a constaté (message réel, abrégé)	Phénomène
1	parReference [2]	(1, 1e8, 1)	y = −7.45E-9, RA = −1.0E-8, N = 0.2549 > Tol = 5.0E-15	élimination
2	parReference [3]	(1, 1e200, 1)	y = −Infinity, RA = −1.0E200, N = Infinity	débordement
3	parReference [4]	(1e200, −3e200, 2e200)	y = NaN, RA = 1.0, N = NaN	∞ − ∞
4	parReference [5]	(1e−200, 1, 1e−200)	y = 0.0, RA = −1.0E-200, N = 1.0	absorption
5	residu [1]	(1, 1e8, 1)	r = −7.45E-9 : résidu 0.2549 > 3.1E-15	élimination
6	viete [1]	(1, 1e8, 1)	produit 0.745 au lieu de 1.0 (écart 25,49 %)	élimination
7	construction [1]	racines −1e8 et −1e−8	−1.49E-8 au lieu de −1.0E-8 (écart 49 %)	élimination
8-13	combinaisons [10, 12, 22, 24, 34, 36]	b = 1e8 et c ≠ 0	ex. [10] : x = 4.098E-8 : résidu 0.0978 > 1.44E-14	élimination
14-17	constructionGeneree cas 9, 12, 17, 19	racines très éloignées	ex. cas 9 : 5.999999999617422E-8 au lieu de 6.0E-8	élimination
18	differentiel [1]	(1, 1e8, 1)	−7.45E-9 au lieu de −1.0E-8 (écart 25,49 %)	élimination

Total : 4 + 1 + 1 + 1 + 6 + 4 + 1 = **18**. Tous sont des **FAIL** (assertion rejetée), aucun **ERROR** : grâce à la vérification préalable du nombre de racines, aucun test ne plante.

8.6 Le tableau « détecte / ne détecte pas »

Oracle	Détecte la version naïve ?	Preuve / explication
Référence DE / RA / Tol	Oui, 4 cas sur 8	cas 2 (N = 0,255), 3 (−∞), 4 (NaN), 5 (0.0 au lieu de -10^{-200})
Référence, cas 1, 6, 7, 8	Non	la formule naïve est juste sur ces entrées
Contrat (a = 0, Δ < 0)	Non	les deux versions ont le même comportement
Résidu (erreur inverse)	Oui	(1, 10^8, 1) : résidu 0,255 > $3{,}1\cdot10^{-15}$
Viète	Oui	produit 0,745 au lieu de 1 (la somme, elle, ne voit rien)
Métamorphique : échelle $\times 2^{10}$	Non	fausse mais cohérente : même erreur à toutes les échelles

Oracle	Détecte la version naïve ?	Preuve / explication
Métamorphique : opposé	Non	idem : l'erreur est symétrique
Construction	Oui	racines choisies (-10^8, -10^{-8})
Combinaisons (36 cas)	Oui, 6 sur 36	exactement les cas b = 10^8 et c ≠ 0
Tests dynamiques (20 cas)	Oui, 4 sur 20	cas 9, 12, 17, 19 : racines d'ordres de grandeur très différents
Différentiel BigDecimal	Oui	(1, 10^8, 1)

> **Conclusion à savoir dire**
>
> Un test « (1, −3, 2) → [1, 2] » ne distingue pas les deux versions. Les relations métamorphiques ne voient pas une version fausse mais cohérente. **Aucun oracle ne voit tout** : il faut combiner des oracles partiels (référence + propriétés + construction + différentiel), avec une tolérance relative justifiée par κ·ε.

8.7 Tester une troisième implémentation (celle d'un camarade)

C'est l'usage demandé par l'enseignant : écrire les tests à partir du **prototype**, puis y brancher n'importe quelle implémentation. Trois étapes :

1. copier la classe du camarade dans `src/main/java/inf563/solveur/` (ex. `SolveurCamarade.java`) ; si son paquetage est différent, adapter la ligne `package` ;
2. créer `src/test/java/inf563/solveur/SolveurCamaradeTest.java` :

```java
package inf563.solveur;

class SolveurCamaradeTest extends SuiteOraclesSolveur {
    @Override
    double[] g(double a, double b, double c) {
        return SolveurCamarade.racines(a, b, c);
    }
}
```

3. lancer cette classe dans le Test Explorer : les 84 tests s'exécutent contre son code, et chaque rouge indique précisément le cas et l'écart.

Condition : sa fonction doit respecter le même contrat (racines triées, tableau vide si Δ < 0, exception si a = 0). Si elle renvoie par exemple `null` au lieu d'un tableau vide, les tests concernés donneront ERROR (`NullPointerException`) : la suite révèle aussi les écarts au contrat.

8.8 Et si… ? (questions de compréhension)

Modification	Conséquence
On retire `@Tag("naif")`	`mvnw test` exécute les deux classes (168 tests) : 18 échecs, `BUILD FAILURE`
On lance tout le projet depuis VS Code	168 tests : 150 verts, 18 rouges (tous dans « Solveur NAÏF »)
On rend `SuiteOraclesSolveur` concrète	impossible sans donner un corps à `g` ; la classe n'a pas de sens seule
Les tests appellent `Solveur.racines` au lieu de `g`	`SolveurNaifTest` testerait en fait la version stable : 0 échec, conclusion fausse
On remplace `assertTrue(n <= tol)` par `assertFalse(n > tol)`	le cas NaN passerait (NaN > tol est faux) : 17 échecs au lieu de 18, faux négatif
On ajoute le cas (1, 10^5, 1) avec Tol = 10^{-6} (exercice b du README)	la version naïve **passe** (erreur $3{,}4 \cdot 10^{-7} \le 10^{-6}$) alors que son erreur est près de 10^9 fois supérieure à κ·ε : « Tol trop large laisse passer les bugs »
On remplace l'erreur relative par `|y − RA| / max(1, |RA|)` (exercice c)	17 échecs au lieu de 18 : le cas « absorption » devient vert malgré 100 % d'erreur (piège de la diapo 70)

9. Exécuter, lire et déboguer les tests

> Cette partie décrit ce qui se passe réellement, étape par étape, entre le moment où vous lancez les tests et l'affichage du verdict, puis donne le mode d'emploi pratique dans VS Code et en ligne de commande.

9.1 Ce qui se passe quand on tape .\mvnw.cmd test

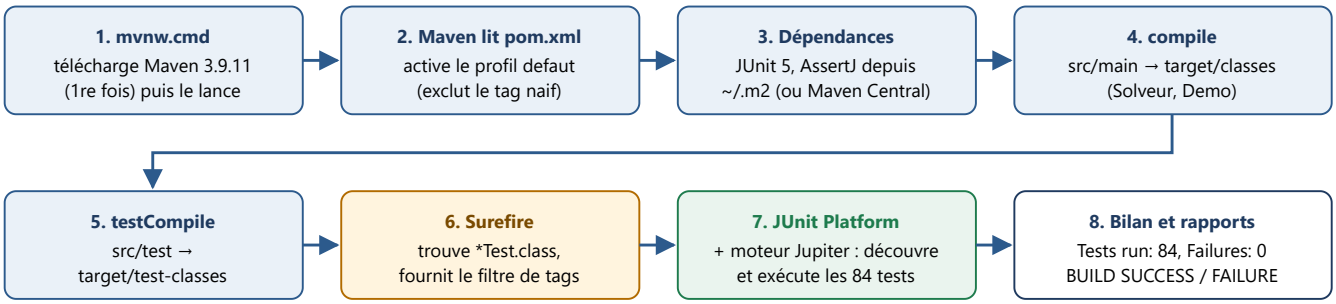


Figure 5 : la chaîne d'exécution de « mvnw test », de la commande jusqu'au verdict global.

Sortie réelle (lignes principales) sur votre machine :

```
[INFO] Building solveur-second-degre 1.0
[INFO] --- compiler:3.14.0:compile (default-compile) @ solveur-second-degre ---
[INFO] --- compiler:3.14.0:testCompile (default-testCompile) @ solveur-second-degre ---
[INFO] --- surefire:3.5.3:test (default-test) @ solveur-second-degre ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] Running inf563.solveur.SolveurStableTest
[INFO] Tests run: 84, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.362 s -- in
inf563.solveur.SolveurStableTest
[INFO] Tests run: 84, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  2.464 s
```

On reconnaît chaque phase du cycle de vie : `compile`, `testCompile`, puis `test` exécuté par Surefire, qui a **détecté automatiquement** JUnit 5 (« JUnitPlatformProvider »). Seule `SolveurStableTest` apparaît : `SolveurNaifTest` a été écartée par le filtre de tags.

9.2 Ce que fait le moteur Jupiter pour un test paramétré

1. **Découverte** : par réflexion, il trouve `parReference`, annotée `@ParameterizedTest`, dans la classe `SolveurStableTest` (méthode héritée).
2. **Filtrage** : la Plateforme compare les tags de la classe au filtre fourni (ici : pas de tag `naif` → gardée).
3. **Source d'arguments** : il lit le `@CsvSource`, découpe chaque ligne en 7 colonnes de texte.
4. **Nom** : il construit le nom affiché avec le modèle `name = "[{index}] {6} : …"`.
5. **Conversion** : il convertit chaque texte vers le type du paramètre (`"1e8"` → 1.0E8).
6. **Instance** : il crée un objet `SolveurStableTest` neuf.
7. **Invocation** : il appelle la méthode avec les 7 arguments.
8. **Verdict** : fin normale → réussi ; `AssertionError` → échec (FAIL) ; autre exception → erreur (ERROR) ; `TestAbortedException` (hypothèse non remplie) → ignoré (INCONCLUSIF).
9. **Recommencer** pour chaque ligne : chaque cas est un test indépendant, avec son propre verdict.

9.3 Lire les résultats

Où	Ce qu'on y trouve
Console Maven	bilan `Tests run / Failures / Errors / Skipped`, puis la liste des échecs avec leur message et la ligne fautive (ex. `SuiteOraclesSolveur.parReference:77`)
`target/surefire-reports/*.txt`	rapport lisible par classe, avec la pile d'appels de chaque échec
`target/surefire-reports/TEST-*.xml`	rapport au format JUnit XML, lu par les outils d'intégration continue (GitHub Actions, GitLab CI, SonarQube)
VS Code, Test Explorer	arbre classe → méthode → cas, icône verte / rouge, message d'échec au clic, durée
Code de retour de `mvnw`	0 si succès, ≠ 0 sinon : c'est ce qui fait échouer un pipeline de CI

9.4 Dans VS Code, pas à pas

1. **Ouvrir le dossier** : *Fichier → Ouvrir le dossier…* → `Projet_test_polynome_chapitre3`. Répondre **Oui** à « Faites-vous confiance aux auteurs ? ».
2. **Attendre « Java: Ready »** en bas à droite (30 s à 2 min la première fois : VS Code importe le projet Maven).
3. **Ouvrir le Test Explorer** : icône **fiole** dans la barre de gauche. On voit `solveur-second-degre → inf563.solveur → Solveur STABLE… / Solveur NAÏF…`. Si une classe n'affiche pas encore ses tests hérités, la lancer une fois avec ▶ : ils apparaissent après l'exécution.
4. **Solveur STABLE** → ▶ *Run Test* : 84 verts.
5. **Solveur NAÏF** → ▶ : 66 verts, 18 rouges (voulu). Cliquer sur un rouge affiche le message de l'oracle, par exemple `N = 0.2549 > Tol = 5.0E-15`.
6. **Déplier un test paramétré** : chaque ligne du CSV est un cas (« [2] élimination catastrophique… »), relançable seul.

9.5 Déboguer un cas qui échoue

1. Ouvrir `Solveur.java` et cliquer dans la marge à gauche de la ligne `double r2 = (-b + Math.sqrt(d)) / (2 * a);` de `naif` : un point rouge (point d'arrêt) apparaît.
2. Dans le Test Explorer, clic droit sur « [2] élimination catastrophique » de *Solveur NAÏF* → **Debug Test**.
3. L'exécution s'arrête sur la ligne : le panneau *Variables* montre `d = 9.999999999999996E15` et `r1 = -1.0E8`. Exécuter la ligne (F10) : `r2 = -7.450580596923828E-9`. Vous **voyez** l'élimination.

9.6 Lancer le programme Demo

Ouvrir `Demo.java` et cliquer sur **Run** au-dessus de `main` (ou en terminal : `java -cp target\classes inf563.solveur.Demo` après un `mvnw test`). Le tableau du § 6.5 s'affiche. Rappel : c'est une démonstration, pas un test.

9.7 Commandes utiles (terminal VS Code : Ctrl + ù)

Commande	Effet
`.\mvnw.cmd test`	compile et exécute la version stable : 84 tests, `BUILD SUCCESS`
`.\mvnw.cmd test -Pnaif`	exécute la version naïve : 84 tests, 18 échecs, `BUILD FAILURE` (attendu)
`.\mvnw.cmd clean test`	supprime `target/` puis recompile tout : utile en cas de doute
`.\mvnw.cmd test -Dtest=SolveurStableTest#viete`	n'exécute qu'une méthode de test
`java -version`	vérifie que Java 21 est bien trouvé

9.8 Exercices pour s'approprier le modèle

a. **Ajouter un cas = ajouter une ligne.** Dans le `textBlock` de `parReference`, ajouter `1, 1e5, 1, -99999.99999, -1.0000000001e-5, 5e-15, b = 1e5` et la ligne équivalente pour b = 10^6. La version stable reste verte, la version naïve échoue sur les deux (erreurs $3{,}4 \cdot 10^{-7}$ et $7{,}6 \cdot 10^{-6}$).

b. **Une tolérance trop large cache les bugs.** Mettre Tol = `1e-6` (celle de la diapo 69) sur ces deux lignes : la version naïve **passe** pour b = 10^5. Remettre `5e-15` ensuite.

c. **Une tolérance absolue masque 25 % d'erreur.** Dans `erreurRelative`, remplacer `/ Math.abs(ra)` par `/ Math.max(1, Math.abs(ra))` : 17 échecs au lieu de 18, le cas « absorption » devient vert alors que la version naïve renvoie 0.0 au lieu de -10^{-200}. Annuler ensuite (Ctrl + Z).

9.9 Dépannage

Problème	Solution
Le Test Explorer est vide	Ctrl + Maj + P → **Java: Clean Java Language Server Workspace** → *Restart and delete*
`java` n'est pas reconnu	fermer et rouvrir VS Code (le PATH contient déjà le JDK 21)
Mauvaise version de Java	Ctrl + Maj + P → **Java: Configure Java Runtime** → choisir JDK 21
`mvnw.cmd` : erreur de téléchargement	vérifier la connexion Internet (premier lancement seulement)
Message « Picked up JAVA_TOOL_OPTIONS… »	informatif : une variable d'environnement force l'UTF-8 de la console ; sans conséquence
Les tests NAÏF sont rouges	normal : c'est la démonstration de ce que les oracles détectent

9.10 Le rendu sous Git

```
git init
git add .
git commit -m "TP séance 3 : oracles du solveur du second degré"
```

Le fichier `.gitignore` exclut `target/` (régénérable), l'enregistrement audio et les PDF du cours (lourds, et qui ne vous appartiennent pas). Une exception `!doc/*.pdf` garde ce document dans le dépôt.

10. Bilan : résultats, limites et suites possibles

10.1 Résultats par bloc de tests

Bloc	Cas	Stable : PASS	Naïf : PASS	Naïf : FAIL
Oracle par référence	8	8	4	4
Contrat (a = 0, Δ < 0)	4	4	4	0
Résidu	3	3	2	1
Viète	2	2	1	1
Métamorphiques (échelle, opposé)	5	5	5	0
Construction	3	3	2	1
Combinaisons	36	36	30	6
Tests dynamiques	20	20	16	4
Différentiel	3	3	2	1
Total	**84**	**84**	**66**	**18**

Lecture : la version stable satisfait **tous** les oracles, avec des tolérances de l'ordre de κ·ε. La version naïve est détectée par sept blocs sur neuf (seuls le contrat et les relations métamorphiques ne la voient pas) ; tous ses échecs relèvent des quatre pièges numériques du chapitre II.

10.2 Les limites du projet (à reconnaître honnêtement)

Limite	Explication
RA calculés hors du projet	Les 8 RA viennent d'un calcul Python à 1 200 chiffres qui n'est pas inclus dans le dépôt. On pourrait les recalculer dans le test en `BigDecimal`, comme le fait `differentiel`.
Entrées spéciales non spécifiées	Le contrat ne dit rien de `racines(NaN, 1, 1)` ou de coefficients infinis : ces cas ne sont pas testés. Les classes d'équivalence du chapitre IV conduiraient à les spécifier puis à les tester.
Tolérances en pourcentage	Viète (10^{-12}), construction (10^{-11}) et différentiel (10^{-14}) utilisent une tolérance fixe, choisie avec marge pour des cas bien conditionnés, au lieu d'un c·κ·ε calculé cas par cas.
Racine double et égalité `d == 0`	Une racine « presque double » donne deux racines très proches plutôt qu'une : c'est conforme au contrat, mais le contrat lui-même pourrait préciser ce cas.
Écarts de coefficients extrêmes	Si les coefficients diffèrent de plus de ~10^{308}, la mise à l'échelle rend le plus petit dénormalisé (perte de précision), voire nul ; ce cas extrême n'est pas couvert.
Build rouge avec `-Pnaif`	Voulu pour la démonstration, mais on ne lancerait pas ce profil dans une intégration continue.

10.3 Ce que les chapitres suivants ajouteraient

Chapitre (séance)	Apport possible au projet
IV : boîte noire	classes d'équivalence (signe de Δ, signe de a, b = 0, c = 0, coefficients spéciaux) et valeurs limites (MIN_VALUE, MAX_VALUE, sous-normaux) pour choisir les cas de façon systématique
V : boîte blanche, couverture, mutation	JaCoCo pour mesurer la couverture des branches de `naif` et `stable` ; PIT pour générer automatiquement des mutants et mesurer le score de mutation de la suite
VI : vérification des calculs	solutions manufacturées (généralisation de l'oracle par construction), analyse d'erreur plus fine
IX à XII : automatisation, CI	pipeline GitHub Actions ou GitLab CI qui exécute `mvnw test` à chaque commit, rapports JUnit XML publiés

11. Questions probables de l'enseignant et réponses

> Les réponses sont formulées comme on les dirait à l'oral : courtes, précises, avec un exemple chiffré quand c'est possible. Entre parenthèses, la section du document où le sujet est approfondi.

11.1 Le fonctionnement général du projet

Q1. Que fait votre projet, en une phrase ?

Il teste automatiquement avec JUnit 5 une fonction qui résout $a \cdot x^2 + b \cdot x + c = 0$, en appliquant le modèle du chapitre III (DE, g, oracle, RA, N, Tol, verdict). La même suite de 84 tests valide la version stable (84/84) et détecte 18 défauts dans la version naïve. (§ 1)

Q2. Pourquoi deux versions du solveur ?

Parce qu'une suite de tests ne vaut que si elle sait échouer. La version naïve contient des défauts numériques réels (diapos 51-52) ; la faire tester par la même suite prouve que nos oracles les détectent. C'est aussi la consigne du TP : exécuter la suite contre les deux versions et remplir le tableau « détecte / ne détecte pas ». (§ 8.4)

Q3. Quel est le système sous test ?

La fonction `Solveur.racines(a, b, c)` (version stable) ou `Solveur.racinesNaif(a, b, c)`. Dans les tests, elle est représentée par la méthode abstraite `g(a, b, c)`, le « g » du modèle du cours. (§ 3.6)

Q4. Quel prototype testez-vous et pourquoi est-ce important ?

`double[] racines(double a, double b, double c)`, avec son contrat : exception si a = 0, tableau vide si $\Delta < 0$, une racine si $\Delta = 0$, deux racines triées sinon. Les tests ne dépendent que de ce prototype, pas du code : ce sont des tests boîte noire, et n'importe quelle implémentation qui respecte le contrat peut être testée, y compris celle d'un camarade. (§ 8.7)

Q5. Comment arrivez-vous à 84 tests ?

11 méthodes de test : 1 `@Test` (1 cas), 9 `@ParameterizedTest` (8 + 3 + 3 + 2 + 3 + 2 + 3 + 36 + 3 = 63 cas) et 1 `@TestFactory` (20 tests dynamiques) : 1 + 63 + 20 = 84. (§ 5.3)

Q6. Comment savez-vous que vos tests sont bons ?

Cinq arguments : ils échouent sur la version naïve (18 échecs couvrant les quatre pièges numériques) ; les tolérances sont justifiées par $c \cdot \kappa \cdot \varepsilon$ et non choisies au hasard ; plusieurs oracles indépendants se recoupent ; les RA sont calculés indépendamment, en haute précision ; tout est reproductible (graine fixe, égalités strictes là où l'arithmétique est exacte). (§ 7, § 8)

Q7. Pourquoi Demo n'est-elle pas un test ?

Elle affiche des résultats sans rien décider : c'est l'humain qui doit lire et juger (oracle humain, coûteux, non automatisable). Un test décide seul avec un critère écrit, comme $N \leq Tol$, et renvoie un verdict. Comme vous l'avez dit en cours : afficher, ce n'est pas tester. Demo n'a d'ailleurs aucune annotation de test et n'est pas exécutée par `mvnw test`. (§ 6.5)

Q8. Quels sont les fichiers principaux et leur rôle ?

`Solveur.java` : le système sous test (deux algorithmes). `SuiteOraclesSolveur.java` : toute la suite de tests, abstraite. `SolveurStableTest` et `SolveurNaifTest` : choisissent l'implémentation testée. `pom.xml` : Java 21, dépendances JUnit 5 et AssertJ, plugins, profils. `mvnw.cmd` : lance Maven sans l'installer. (§ 3.2)

Q9. Comment les fichiers communiquent-ils ?

> Maven (ou VS Code) lance la JUnit Platform ; le moteur Jupiter instancie `SolveurStableTest` et appelle une méthode de test héritée de la suite ; celle-ci appelle `g`, que la liaison dynamique résout vers la version de la sous-classe, qui appelle `Solveur.racines` ; le résultat revient aux assertions qui produisent le verdict. Le solveur, lui, ne connaît pas les tests. (§ 3.6)

11.2 Les bibliothèques et les outils

Q10. Qu'est-ce que JUnit et pourquoi JUnit 5 ?

> JUnit est le framework de tests unitaires de référence en Java : il découvre les tests, les exécute, et rend un verdict par test. JUnit 5 apporte les tests paramétrés (`@ParameterizedTest`), les tests dynamiques (`@TestFactory`), les noms lisibles (`@DisplayName`) et les tags. C'est aussi l'outil prévu par le cours (diapo 6). (§ 4.6)

Q11. Quelle différence entre JUnit Platform, Jupiter et Vintage ?

> La Platform est le socle : elle découvre, filtre et lance les tests pour les outils (Maven, VS Code). Jupiter est le modèle de programmation de JUnit 5 (annotations, assertions) et son moteur ; tout le projet est écrit en Jupiter. Vintage exécute les anciens tests JUnit 3/4 ; il n'est pas utilisé. (§ 4.6)

Q12. Pourquoi AssertJ en plus de JUnit ?

> Parce que `assertEquals(attendu, obtenu, delta)` de JUnit n'accepte qu'un delta absolu, alors que nos racines vont de 10^{-200} à 10^{200} : il faut une comparaison relative. AssertJ fournit `isCloseTo(…, withinPercentage(…))`, une syntaxe lisible et des messages d'échec détaillés. (§ 4.7)

Q13. Que signifie withinPercentage(1e-9) ?

> Que l'on accepte $|\text{obtenu} - \text{attendu}| \leq (10^{-9} / 100) \times |\text{attendu}|$, soit une erreur relative de 10^{-11}. C'est un pourcentage, d'où le facteur 100. (§ 4.7)

Q14. À quoi servent Maven et le pom.xml ?

> Maven construit le projet : il télécharge les dépendances déclarées dans le `pom.xml`, compile le code puis les tests, et exécute les tests pendant la phase `test` de son cycle de vie. Le `pom.xml` décrit l'identité du projet (GAV), la version de Java, les dépendances, les plugins et les profils. (§ 4.3)

Q15. Pourquoi la dépendance junit-jupiter n'a-t-elle pas de version ?

> Parce que la version vient du **BOM** `junit-bom` importé dans `dependencyManagement`. Le BOM fixe d'un coup des versions compatibles pour tous les modules JUnit (API, paramètres, moteur, plateforme) et évite les mélanges de versions. (§ 4.3)

Q16. Que signifie scope test ?

> Que JUnit et AssertJ ne sont disponibles que pour compiler et exécuter le code de `src/test`. Ils ne font pas partie du programme livré et le code de production ne peut pas en dépendre. (§ 4.3)

Q17. Qu'est-ce que Surefire ?

> Le plugin Maven qui exécute les tests pendant la phase `test`. Il cherche les classes dont le nom suit `*Test`, `Test*`, `*Tests` ou `*TestCase`, détecte JUnit 5, applique le filtre de tags, affiche le bilan, écrit les rapports dans `target/surefire-reports` et fait échouer le build en cas de FAIL ou d'ERROR. (§ 4.5)

Q18. À quoi sert le Maven Wrapper ?

> À lancer la bonne version de Maven (3.9.11) sans l'installer : `mvnw.cmd` la télécharge la première fois puis la lance. Tout le monde utilise ainsi la même version. La version 3.3.4 du wrapper a été choisie car la 3.3.2 échouait à cause de l'espace dans « SmartCamp PC1 ». (§ 4.4)

Q19. VS Code utilise-t-il Maven pour lancer les tests ?

> Il lit le `pom.xml` pour connaître le projet et ses dépendances, mais il compile avec son propre compilateur et lance les tests directement via la JUnit Platform, sans Surefire. Les profils et le filtre de tags du `pom.xml` ne s'appliquent donc pas : on peut lancer `SolveurNaifTest` depuis VS Code. (§ 4.9)

Q20. Pourquoi BigDecimal, et pourquoi new BigDecimal(a) plutôt que BigDecimal.valueOf(a) ?

> `BigDecimal` calcule en décimal avec la précision voulue : exactement pour `multiply` et `subtract`, à 50 chiffres pour `sqrt` et `divide`. `new BigDecimal(a)` prend la valeur **exacte** du double (0.1 → 0.1000000000000000055511…), c'est-à-dire exactement l'entrée reçue par le solveur ; `valueOf` partirait de l'écriture décimale arrondie « 0.1 », ce qui serait un autre problème. (§ 4.8)

11.3 Les annotations « @ »

Q21. Qu'est-ce qu'une annotation et que fait-elle concrètement ?

> Une métadonnée attachée au code. Elle ne fait rien par elle-même : elle est lue par un outil. `@Override` est lue par le compilateur ; les annotations JUnit sont lues à l'exécution par réflexion. Elles indiquent à JUnit quelles méthodes sont des tests, d'où viennent leurs données et comment les nommer ou les filtrer. (§ 5.1)

Q22. Que se passe-t-il si on oublie @Test ou @ParameterizedTest ?

> Le code compile, mais la méthode devient une méthode ordinaire : JUnit ne l'exécute plus, sans aucun avertissement. Le nombre de tests baisse silencieusement. C'est pourquoi on vérifie toujours le nombre de tests exécutés (`Tests run: 84`). (§ 5.2)

Q23. Quelle différence entre @Test, @ParameterizedTest et @TestFactory ?

> `@Test` : un test, exécuté une fois, sans paramètre. `@ParameterizedTest` : un test exécuté une fois par jeu d'arguments fourni par une source (`@CsvSource`, `@MethodSource`…), cas connus à l'écriture. `@TestFactory` : une méthode qui fabrique des tests à l'exécution et renvoie des `DynamicTest`. (§ 5.3, § 5.6)

Q24. Comment @CsvSource transforme-t-il une ligne en arguments ?

> Chaque ligne est découpée sur les virgules ; la colonne i devient le paramètre i. JUnit convertit le texte vers le type du paramètre (`"1e8"` → double 1.0E8, texte → String). Dans un `textBlock`, les lignes commençant par `#` sont des commentaires. Une ligne = un cas de test = DE + RA + Tol. (§ 5.4)

Q25. Comment obtenez-vous « toutes les combinaisons » des paramètres ?

> Un tableau de valeurs par paramètre (3 pour a, 4 pour b, 3 pour c) et une méthode `static` qui construit leur produit cartésien avec des `flatMap` imbriqués ; `@MethodSource("toutesLesCombinaisons")` la branche sur le test. JUnit exécute 3 × 4 × 3 = 36 cas. (§ 5.5)

Q26. Pourquoi toutesLesCombinaisons est-elle static ?

> Parce que JUnit crée une nouvelle instance de la classe de test pour chaque cas (cycle de vie « par méthode ») : il doit appeler la méthode source **avant** d'avoir une instance. Avec `@TestInstance(PER_CLASS)`, elle pourrait ne pas l'être. (§ 4.6)

Q27. À quoi servent @DisplayName et name = "..." ?

> À afficher des noms lisibles dans VS Code et les rapports : « Oracle par référence : N(y, RA) ≤ Tol » puis « [2] élimination catastrophique : (1, 1e8, 1) → … » au lieu de `parReference(double, double, …)[2]`. `{index}` et `{0}`… sont remplacés par le numéro du cas et ses arguments. (§ 5.7)

Q28. À quoi sert @Tag("naif") ?

> À étiqueter tous les tests de `SolveurNaifTest`. Le profil Maven `defaut` passe `excludedGroups = naif` à Surefire : `mvnw test` n'exécute que la version stable et reste vert. Le profil `naif` (`-Pnaif`) fait l'inverse. L'annotation ne change rien à l'exécution d'un test ; elle ne sert qu'au filtrage. (§ 4.3, § 5.7)

Q29. @Override est-elle obligatoire ?

> Non, mais recommandée : elle demande au compilateur de vérifier que `g` redéfinit bien une méthode de la classe mère, et documente l'intention. Ici une faute de signature serait de toute façon détectée car `g` est abstraite. (§ 5.7)

Q30. Pourquoi vos classes et méthodes de test ne sont-elles pas public ?

> JUnit 5, contrairement à JUnit 4, accepte la visibilité « paquetage ». Seul `private` est interdit (et `static` pour les méthodes de test). Moins de mots-clés, même comportement. (§ 3.4)

11.4 SolveurNaifTest et SolveurStableTest

Q31. Quelle est la différence entre SolveurNaifTest et SolveurStableTest ?

> Une seule ligne de code utile : leur méthode `g` appelle `Solveur.racines` (stable) ou `Solveur.racinesNaif` (naïve). Elles héritent de la même suite, donc des mêmes 84 tests. En plus, la classe naïve porte `@Tag("naif")` pour être exclue du build par défaut. Résultats : 84 verts pour la stable, 66 verts et 18 rouges pour la naïve. (§ 8.3)

Q32. Pourquoi n'avoir écrit les tests qu'une fois ?

> Pour juger les deux versions avec exactement les mêmes données, oracles et tolérances (comparaison équitable), et pour pouvoir tester une nouvelle implémentation en ajoutant une classe de 6 lignes. C'est le patron Méthode patron (*Template Method*) : la classe abstraite fixe l'algorithme de test, les sous-classes fournissent l'étape variable `g`. (§ 3.7)

Q33. Vos tests naïfs sont rouges : votre projet échoue-t-il ?

> Non. Le produit livré est la version stable, verte à 100 %. Les 18 rouges sont attendus : ils prouvent que la suite détecte l'élimination catastrophique, le débordement, le NaN et l'absorption. Un test qui ne peut jamais échouer ne sert à rien. (§ 8.4)

Q34. Pourquoi mvnw test n'exécute-t-il pas SolveurNaifTest alors que VS Code le peut ?

> Le filtre de tags est configuré dans Surefire, via le profil Maven actif par défaut. VS Code n'utilise pas Surefire : il lance directement la JUnit Platform, sans ce filtre. (§ 4.9)

Q35. Quels oracles ne détectent pas la version naïve, et pourquoi ?

> Les relations métamorphiques (échelle $\times 2^{10}$, opposé) et le contrat. La version naïve est « fausse mais cohérente » : sur $(1024, 1024 \cdot 10^8, 1024)$, tous les calculs sont multipliés par des puissances de 2 exactes, elle refait la même erreur ; une relation entre deux résultats faux de la même façon reste vraie. D'où la nécessité de combiner les oracles. (§ 7.5)

Q36. Comment testeriez-vous la fonction d'un camarade ?

> Je copie sa classe dans `src/main/java/inf563/solveur` et j'écris une classe `SolveurCamaradeTest extends SuiteOraclesSolveur` dont `g` appelle sa fonction. Les 84 tests s'exécutent alors contre son code, sans rien modifier d'autre. (§ 8.7)

11.5 Questions techniques : numérique et algorithmes

Q37. Pourquoi la formule du lycée donne-t-elle $-7{,}45 \cdot 10^{-9}$ au lieu de -10^{-8} pour $(1, 10^8, 1)$?

> La vraie valeur de $\sqrt{\Delta}$ est $10^8 - 2 \cdot 10^{-8}$, mais les doubles proches de 10^8 sont espacés de $\mathrm{ulp}(10^8) \approx 1{,}49 \cdot 10^{-8}$: $\sqrt{\Delta}$ est arrondi à $10^8 - 1{,}49 \cdot 10^{-8}$. La soustraction $-b + \sqrt{\Delta}$, exacte, donne alors $-1{,}49 \cdot 10^{-8}$ au lieu de $-2 \cdot 10^{-8}$; divisé par 2 : $-7{,}45 \cdot 10^{-9}$. 25 % d'erreur. (§ 2.3)

Q38. Qu'est-ce que l'élimination catastrophique ?

> La soustraction de deux nombres très proches : leurs chiffres communs s'annulent et il ne reste que les erreurs d'arrondi déjà présentes, qui deviennent dominantes. La soustraction est exacte ; elle révèle l'information perdue avant (diapo 44). Remède : réécrire la formule pour éviter la soustraction. (§ 2.3)

Q39. Pourquoi b^2 déborde-t-il alors que les racines sont représentables ?

> Pour $b = 10^{200}$, $b^2 = 10^{400}$ dépasse le plus grand double ($\approx 1{,}8 \cdot 10^{308}$) et devient $+\infty$. Les racines (-10^{200} et -10^{-200}) sont représentables : c'est un calcul **intermédiaire** qui déborde. La mise à l'échelle par 2^{-e} l'empêche. (§ 2.5)

Q40. D'où vient le NaN, et pourquoi aucun if ne l'arrête ?

> Pour (10^{200}, $-3 \cdot 10^{200}$, $2 \cdot 10^{200}$), b^2 et $4ac$ valent tous deux $+\infty$, et $\infty - \infty$ = NaN. Or toute comparaison avec NaN est fausse : `d < 0` et `d == 0` sont faux, le code continue, et $\sqrt{\text{NaN}}$ = NaN se propage jusqu'au résultat, sans exception. (§ 2.2)

Q41. Qu'est-ce que l'absorption ?

> Quand on ajoute un nombre très petit à un nombre beaucoup plus grand, le petit disparaît : $1 - 4 \cdot 10^{-400} = 1$ en double (ici $4ac$ est même d'abord arrondi à 0). Pour (10^{-200}, 1, 10^{-200}), la petite racine devient 0 au lieu de -10^{-200}. (§ 2.3)

Q42. Expliquez la formule stable.

> On calcule $q = -\frac{1}{2}(b + \text{signe}(b) \cdot \sqrt{\Delta})$: b et $\text{signe}(b) \cdot \sqrt{\Delta}$ ont le même signe, donc on additionne sans élimination. $r_1 = q/a$ est la racine « sûre » ; $r_2 = c/q$ vient de Viète ($r_1 \cdot r_2 = c/a$), par une simple division. En amont, on divise a, b, c par 2e pour éviter les débordements. (§ 2.4)

Q43. Pourquoi Math.copySign ?

> `Math.copySign(Math.sqrt(d), b)` renvoie $\sqrt{\Delta}$ avec le signe de b. C'est la façon exacte et sans branche d'écrire « $\text{signe}(b) \cdot \sqrt{\Delta}$ », qui garantit que b et ce terme s'additionnent sans se compenser. (§ 2.4)

Q44. Pourquoi diviser par une puissance de 2 plutôt que par le plus grand coefficient m ?

> Diviser par m (quelconque) introduirait une erreur d'arrondi sur chaque coefficient. Multiplier par 2^{-e} ne change que l'exposant binaire : c'est exact. On obtient le même effet (plus grand coefficient dans [1, 2[, donc pas de débordement) sans aucune erreur. (§ 2.5)

Q45. Que sont ε et l'ulp ?

> ulp(x) (*unit in the last place*) est l'écart entre x et le double suivant. $\varepsilon = \text{ulp}(1) = 2^{-52} \approx 2{,}22 \cdot 10^{-16}$ est l'epsilon machine : l'erreur relative maximale d'un arrondi est ε/2. Dans le code : `EPS = Math.ulp(1.0)`. (§ 2.2)

Q46. Qu'est-ce que le conditionnement κ ? Calculez-le pour (1, −3, 2).

> La sensibilité de la solution exacte aux petites perturbations des données ; c'est une propriété du problème, pas de l'algorithme. $\kappa = (|a|r^2 + |b||r| + |c|) / (|r| \cdot |2ar + b|)$. Pour r = 1 : (1 + 3 + 2) / (1·|2 − 3|) = 6 ; pour r = 2 : (4 + 6 + 2) / (2·|4 − 3|) = 6. (§ 2.6)

Q47. Pourquoi le cas $\Delta \approx 0$ a-t-il une tolérance de $5 \cdot 10^{-7}$? N'est-ce pas trop large ?

> Non, elle est justifiée : la racine est presque double, $\kappa \approx 1{,}9 \cdot 10^8$, et $10 \cdot \kappa \cdot \varepsilon \approx 4{,}2 \cdot 10^{-7}$. Aucun algorithme en double ne peut faire mieux qu'environ $\sqrt{\varepsilon} \approx 10^{-8}$ en relatif ; c'est le problème qui est mal conditionné. Les deux versions donnent d'ailleurs exactement le même résultat. Exiger 10^{-15} créerait un faux échec. (§ 2.6)

Q48. Pourquoi ne jamais écrire assertEquals(2.0, y) sur un double calculé ?

> Parce qu'un résultat flottant porte presque toujours une erreur d'arrondi : 0.1 + 0.2 vaut 0.30000000000000004. Il faut une tolérance justifiée. Exception : quand l'arithmétique est exacte (multiplication par 2^{10}, changement de signe), l'égalité stricte est légitime, comme dans `echelle` et `oppose`. (§ 7.5)

Q49. Pourquoi Tol = 0 pour la racine double (1, −2, 1) ?

> Parce que tous les calculs y sont exacts : $b^2 = 4$ et $4ac = 4$ sont des petits entiers représentables, donc $\Delta = 0$ exactement, et $-b/2a = 1$ exactement. On peut exiger le résultat exact. (§ 2.6)

11.6 Les oracles et le chapitre III

Q50. Qu'est-ce qu'un oracle de test ?

> Le mécanisme qui décide si la sortie y observée pour l'entrée x est acceptable : un prédicat O(x, y) → vrai/faux. Il peut être par référence (N(y, RA) ≤ Tol), par propriété, métamorphique, différentiel, implicite ou humain. Dans le code, ce sont les assertions. (§ 2.7)

Q51. Qu'est-ce que le problème de l'oracle ?

> Le fait qu'on ne connaisse souvent pas le résultat attendu, ou qu'il soit trop coûteux de vérifier une sortie (Weyuker, 1982). C'est le cas en calcul scientifique. On le contourne en combinant des oracles partiels qui n'ont pas besoin de RA : résidu, Viète, relations métamorphiques, construction, différentiel. (§ 2.9)

Q52. Quels sont les quatre verdicts et leur équivalent JUnit ?

> PASS : test vert, y compris une exception attendue (`assertThrows`). FAIL : une assertion échoue (`Failures`). ERROR : exception imprévue (`Errors`). INCONCLUSIF : hypothèse non remplie, environnement défaillant (`Skipped`, via `Assumptions`). (§ 2.7)

Q53. Pourquoi l'erreur relative et pas l'erreur absolue ?

> Les racines testées vont de 10^{-200} à 10^{200} : aucune tolérance absolue n'a de sens à toutes ces échelles. Une tolérance absolue de 10^{-6} accepterait 0 pour -10^{-200} (100 % d'erreur), et `max(1, |RA|)` masque 25 % d'erreur sur (1, 10^8, 1) (diapo 70). Aucun de nos RA n'est nul, donc l'erreur relative est toujours définie. (§ 2.8)

Q54. Comment justifiez-vous vos tolérances ?

> Par la règle Tol = c·κ·ε avec c = 10 : ε est la précision machine, κ le conditionnement de la racine, calculé pour chaque cas. Exemple : élimination, κ = 2, $10 \cdot \kappa \cdot \varepsilon = 4{,}4 \cdot 10^{-15}$, Tol retenue $5 \cdot 10^{-15}$. Le tableau complet est dans `RAPPORT.md`. (§ 2.6)

Q55. Qu'est-ce qu'une relation métamorphique ? Pourquoi k = 1024 ?

> Une relation entre plusieurs exécutions (T. Y. Chen, 1998) : on ne connaît pas y, mais on sait comment y doit varier quand x varie. Ici : multiplier a, b, c par k ne change pas les racines ; remplacer b par −b les change en leurs opposées. k = 1024 = 2^{10} rend la multiplication exacte, donc on peut exiger l'égalité stricte ; avec k = 10, les résultats diffèrent au dernier chiffre. (§ 7.5)

Q56. Qu'est-ce que l'erreur inverse mesurée par le test residu ? Pourquoi le facteur 8 ?

> Le résidu $|ar^2 + br + c|$ divisé par l'échelle $|a|r^2 + |br| + |c|$ mesure de combien il faudrait perturber les coefficients pour que r soit une racine exacte. Un bon algorithme donne quelques ε. Le facteur 8 laisse une marge pour les arrondis de l'algorithme et du calcul du résidu lui-même. Sans cette échelle, un résidu dû aux arrondis passerait pour un bug (diapo 78). (§ 7.5)

Q57. Qu'est-ce que l'oracle par construction, et quelle est sa limite ?

> On choisit les racines r_1, r_2, on fabrique l'équation $x^2 - (r_1 + r_2)x + r_1 r_2 = 0$ et on vérifie qu'on retrouve r_1, r_2 : le RA est connu par construction. Limite : b et c sont eux-mêmes arrondis, donc l'équation construite n'a pas exactement r_1, r_2 pour racines ; d'où une tolérance de 10^{-11} plutôt que ε. (§ 7.5)

Q58. Le test différentiel utilise la formule naïve. Pourquoi est-il fiable ?

> Parce qu'il la calcule en `BigDecimal` à 50 chiffres : l'élimination fait perdre environ 16 chiffres, il en reste environ 34, bien plus que les 16 d'un double. Et `new BigDecimal(double)` part des valeurs exactes des entrées. (§ 7.8)

Q59. Pourquoi calculer le signe de Δ en BigDecimal dans le test des combinaisons ?

> Parce que l'oracle ne doit pas être aussi fragile que le programme testé. Calculé en double, Δ pourrait changer de signe à cause d'un arrondi près de 0. En `BigDecimal` sans contexte, `multiply` et `subtract` sont exacts : le signe est certain, et `signum() + 1` donne le nombre de racines attendu. (§ 7.6)

Q60. Qu'est-ce qu'un faux positif et un faux négatif ?

> Faux positif : le test échoue alors que le logiciel est correct (tolérance trop étroite, oracle faux). Faux négatif : le test passe alors que le logiciel est défaillant (tolérance trop large, mesure absolue) : le plus dangereux. Nos tolérances c·κ·ε visent à éviter les deux. (§ 2.6)

11.7 Questions d'ouverture et questions pièges

Q61. Vos tests « aléatoires » sont-ils reproductibles ?

> Oui : `new Random(563)` utilise une graine fixe, donc les 20 cas générés sont toujours les mêmes et un échec se reproduit à l'identique. Changer la graine permet d'explorer d'autres cas. (§ 5.6)

Q62. Pourquoi ne pas tester toutes les valeurs possibles ?

> L'exhaustivité est impossible (principe 2 du test, diapo 18) : il existe 2^{64} doubles, donc $2^{192} \approx 6 \cdot 10^{57}$ triplets (a, b, c). On choisit des cas représentatifs et des cas aux limites (débordement, absorption, $\Delta \approx 0$), et on les multiplie par combinaison et génération.

Q63. Quelles sont les limites de votre projet ?

> Les RA sont calculés hors du projet ; le contrat ne spécifie pas les entrées NaN ou infinies ; certaines tolérances (Viète, construction) sont fixes au lieu d'être calculées par κ ; les écarts extrêmes de coefficients ne sont pas couverts. (§ 10.2)

Q64. Comment mesureriez-vous la qualité de votre suite de tests ?

> Par la couverture de code (JaCoCo : toutes les branches de `naif` et `stable` sont-elles exécutées ?) et surtout par le test de mutation (PIT) : on introduit automatiquement de petits défauts et on compte ceux que la suite détecte. La version naïve joue déjà ce rôle de « mutant » réaliste. (§ 10.3)

Q65. Comment intégreriez-vous ces tests dans une intégration continue ?

> Un pipeline (GitHub Actions, GitLab CI) exécute `./mvnw test` à chaque commit ; le profil par défaut garantit un build vert tant que la version stable est correcte ; les rapports `TEST-*.xml` de Surefire sont publiés. (§ 9.3)

Q66. Si la version stable avait un bug, comment le verriez-vous ?

> Par un ou plusieurs tests rouges dans « Solveur STABLE », avec un message qui donne le cas, la valeur obtenue, la référence et l'écart (N, Tol). Je reproduirais l'échec (même cas, même graine), je le déboguerais avec un point d'arrêt (Debug Test), puis je corrigerais et relancerais toute la suite.

12. Glossaire

Terme	Définition
Absorption	Disparition d'un terme très petit ajouté à un terme beaucoup plus grand ($1 + 10^{-400} = 1$ en double).
Annotation	Métadonnée `@Nom` attachée au code, lue par le compilateur ou par un framework (JUnit) par réflexion.
AssertJ	Bibliothèque d'assertions fluides ; fournit la comparaison relative `withinPercentage`.
Assertion	Instruction qui vérifie une condition et fait échouer le test si elle est fausse : c'est l'oracle dans le code.
BOM	*Bill Of Materials* : liste de versions compatibles importée dans Maven (`junit-bom`).
Classe abstraite	Classe non instanciable, qui peut déclarer des méthodes sans corps (`g`) à fournir par les sous-classes.
Conditionnement κ	Sensibilité de la solution exacte aux perturbations des données ; propriété du problème.
Contrat	Ce que la fonction promet (racines triées, vide si $\Delta < 0$, exception si $a = 0$).
DE	Données d'entrée d'un test : (a, b, c).
Débordement (*overflow*)	Résultat trop grand pour un double : il devient $\pm\infty$.
Dépassement inférieur (*underflow*)	Résultat trop petit : il devient dénormalisé puis 0.
Élimination catastrophique	Perte des chiffres significatifs lors de la soustraction de deux nombres proches.
Epsilon machine ε	`Math.ulp(1.0)` $= 2^{-52} \approx 2{,}22 \cdot 10^{-16}$.
Erreur directe / inverse	Directe : écart entre y et la vraie solution. Inverse : perturbation des données qui ferait de y une solution exacte (résidu / échelle).
Erreur relative	$\lvert y - RA \rvert / \lvert RA \rvert$: la mesure N du projet.
Faux négatif / faux positif	Test qui passe malgré un défaut / test qui échoue sans défaut.
g, SUT	Système sous test (*system under test*) : la fonction exécutée par le test.
GAV	Coordonnées Maven : *groupId, artifactId, version*.
IEEE 754	Norme des nombres flottants (format, arrondi, $\pm\infty$, NaN).
JUnit Platform / Jupiter	Socle de lancement des tests / modèle de programmation et moteur de JUnit 5.
Liaison dynamique	Choix, à l'exécution, de la version de la méthode selon la classe réelle de l'objet (polymorphisme).
Maven, Wrapper	Outil de construction ; script qui télécharge et lance la bonne version de Maven.
Méthode patron (*Template Method*)	Patron où une classe abstraite fixe l'algorithme et délègue une étape aux sous-classes.
N	Mesure d'erreur entre la sortie et le résultat attendu.
NaN	*Not a Number* : résultat d'une opération indéfinie ($\infty - \infty$) ; toute comparaison avec NaN est fausse.
Oracle	Mécanisme qui décide si une sortie est acceptable ; partiel s'il ne vérifie qu'une propriété.
Oracle différentiel	Compare à une autre implémentation (ici BigDecimal 50 chiffres).
Oracle par construction	On choisit la réponse, puis on fabrique le problème correspondant.
Profil Maven	Variante de configuration activable (`-Pnaif`).
RA	Résultat attendu (référence).
Réflexion	Capacité d'un programme Java à inspecter ses classes et méthodes à l'exécution (utilisée par JUnit pour lire les annotations).
Relation métamorphique	Relation entre les sorties de plusieurs exécutions liées (échelle, opposé).
Résidu	$\lvert a \cdot r^2 + b \cdot r + c \rvert$: à quel point r annule le polynôme.
Scope test	Dépendance Maven disponible seulement pour le code de test.
Stabilité	Propriété d'un algorithme qui n'ajoute pas plus d'erreur que le conditionnement n'en impose.
Surefire	Plugin Maven qui exécute les tests pendant la phase `test`.
Tag	Étiquette JUnit (`@Tag`) servant à filtrer les tests.
Test dynamique	Test créé à l'exécution par une méthode `@TestFactory`.
Test paramétré	Test exécuté une fois par jeu d'arguments (`@ParameterizedTest`).
Tol	Tolérance : écart maximal accepté, justifié par $c \cdot \kappa \cdot \varepsilon$.
ulp	*Unit in the last place* : écart entre un double et le double suivant.
Verdict	PASS, FAIL, ERROR ou INCONCLUSIF.
Viète (relations de)	$r_1 + r_2 = -b/a$ et $r_1 \cdot r_2 = c/a$.

Annexes : le code source complet

> Le code ci-dessous est inséré automatiquement depuis les fichiers du projet au moment de la génération de ce document : il est identique à celui que vous avez dans VS Code. Les numéros de ligne correspondent à ceux de l'éditeur et des messages d'échec (ex. `SuiteOraclesSolveur.parReference:77`).

A. src/main/java/inf563/solveur/Solveur.java

```java
package inf563.solveur;

/**
 * Fil rouge A : solveur de l'équation du second degré a·x² + b·x + c = 0.
 *
 * <p>Contrat commun aux deux versions :
 * <ul>
 *   <li>a = 0 : IllegalArgumentException (ce n'est plus une équation du second degré) ;</li>
 *   <li>Δ &lt; 0 : tableau vide (pas de racine réelle) ;</li>
 *   <li>Δ = 0 : une racine double ;</li>
 *   <li>Δ &gt; 0 : deux racines triées dans l'ordre croissant.</li>
 * </ul>
 */
public final class Solveur {

    private Solveur() {
    }

    /** Système sous test g utilisé par défaut : la version stable. */
    public static double[] racines(double a, double b, double c) {
        return stable(a, b, c);
    }

    /** Système sous test g, version naïve (pour montrer ce que les oracles détectent). */
    public static double[] racinesNaif(double a, double b, double c) {
        return naif(a, b, c);
    }

    /**
     * Formule du lycée : (-b ± √Δ) / 2a.
     * Mathématiquement juste, numériquement fragile : élimination catastrophique quand b² ≫ 4ac,
     * débordement de b² en +∞, ∞ − ∞ = NaN, absorption.
     */
    static double[] naif(double a, double b, double c) {
        if (a == 0) throw new IllegalArgumentException("a = 0");
        double d = b * b - 4 * a * c;
        if (d < 0) return new double[0];
        if (d == 0) return new double[]{-b / (2 * a)};
        double r1 = (-b - Math.sqrt(d)) / (2 * a);
        double r2 = (-b + Math.sqrt(d)) / (2 * a);
        return new double[]{Math.min(r1, r2), Math.max(r1, r2)};
    }

    /**
     * Formule stable + mise à l'échelle.
     * q additionne deux termes de même signe (plus de soustraction de nombres proches),
     * la seconde racine vient de Viète (r1·r2 = c/a). Multiplier par 2^k est exact en binaire :
     * les racines ne changent pas et b² ne déborde plus.
     */
    static double[] stable(double a, double b, double c) {
        if (a == 0) throw new IllegalArgumentException("a = 0");
        // mise à l'échelle exacte par une puissance de 2
        double m = Math.max(Math.abs(a), Math.max(Math.abs(b), Math.abs(c)));
        int e = Math.getExponent(m);
        a = Math.scalb(a, -e);
        b = Math.scalb(b, -e);
        c = Math.scalb(c, -e);
        double d = b * b - 4 * a * c;
        if (d < 0) return new double[0];
        if (d == 0) return new double[]{-b / (2 * a)};
        double q = -0.5 * (b + Math.copySign(Math.sqrt(d), b));
        double r1 = q / a, r2 = c / q;          // Viète : r1 * r2 = c / a
        return new double[]{Math.min(r1, r2), Math.max(r1, r2)};
    }
}
```

B. src/main/java/inf563/solveur/Demo.java

```java
package inf563.solveur;

import java.util.Arrays;

/**
 * Affiche le tableau du TP : résultat naïf / stable / attendu pour chaque cas.
 * (Sortie en ASCII pour rester lisible dans la console Windows.)
 */
public class Demo {

    public static void main(String[] args) {
        // a, b, c, diagnostic, résultat attendu
        Object[][] cas = {
                {1, -3, 2, "cas nominal", "[1.0, 2.0]"},
                {1, 1e8, 1, "elimination catastrophique", "[-1.0E8, -1.0E-8]"},
                {1, 1e200, 1, "b^2 deborde en +Inf", "[-1.0E200, -1.0E-200]"},
                {1e200, -3e200, 2e200, "Inf - Inf = NaN", "[1.0, 2.0]"},
                {1e-200, 1, 1e-200, "absorption", "[-1.0E200, -1.0E-200]"},
                {1, -2, 1 - 1.1e-16, "Delta ~ 0 (mal conditionne)", "[0.99999998946..., 1.00000001054...]"},
        };
        String format = "%-28s | %-34s | %-42s | %-42s | %s%n";
        System.out.printf(format, "Diagnostic", "(a, b, c)", "Naif", "Stable", "Attendu");
        System.out.println("-".repeat(190));
        for (Object[] l : cas) {
            double a = ((Number) l[0]).doubleValue(), b = ((Number) l[1]).doubleValue(), c = ((Number) l[2]).doubleValue();
            System.out.printf(format, l[3], "(" + a + ", " + b + ", " + c + ")",
                    Arrays.toString(Solveur.racinesNaif(a, b, c)), Arrays.toString(Solveur.racines(a, b, c)), l[4]);
        }
    }
}
```

C. src/test/java/inf563/solveur/SuiteOraclesSolveur.java

```java
package inf563.solveur;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.withinPercentage;
import static org.junit.jupiter.api.Assertions.assertArrayEquals;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.junit.jupiter.api.DynamicTest.dynamicTest;

import java.math.BigDecimal;
import java.math.MathContext;
import java.util.Arrays;
import java.util.Random;
import java.util.stream.IntStream;
import java.util.stream.Stream;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.DynamicTest;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestFactory;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.Arguments;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.MethodSource;

/**
 * Séance 3 : la même suite de tests, exécutée contre les deux versions du solveur.
 *
 * <p>Modèle d'un test : DE (données d'entrée) → g (système sous test) → y,
 * puis l'oracle O décide PASS / FAIL. Pour l'oracle par référence : N(y, RA) ≤ Tol.
 *
 * <p>Les sous-classes ne font qu'une chose : dire quel g on teste.
 */
abstract class SuiteOraclesSolveur {

    /** ε = ulp(1) = 2^-52 ≈ 2,2·10^-16 en double. */
    static final double EPS = Math.ulp(1.0);

    /** Le système sous test g : renvoie les racines triées de a·x² + b·x + c = 0. */
    abstract double[] g(double a, double b, double c);

    /** Mesure d'erreur N : erreur relative |y − RA| / |RA| (aucun RA n'est nul dans nos cas). */
    static double erreurRelative(double y, double ra) {
        return Math.abs(y - ra) / Math.abs(ra);
```

```java
    }

    // ---------------------------------------------------------------
    // 1. Oracle par référence : DE / RA / Tol (8 cas)
    //    Tol = c·κ·ε avec c = 10 (marge), κ = conditionnement de la racine.
    //    RA calculés à part avec 1 200 chiffres (Python Decimal) à partir
    //    des valeurs exactes des double.
    // ---------------------------------------------------------------
    @ParameterizedTest(name = "[{index}] {6} : ({0}, {1}, {2}) → [{3}, {4}], Tol = {5}")
    //   κ et 10·κ·ε par cas :  nominal κ=6 → 1,3e-14 ; élimination κ=2 → 4,4e-15 ;
    //   débordement κ=2 → 4,4e-15 ; NaN κ=6 → 1,3e-14 ; absorption κ=2 → 4,4e-15 ;
    //   a négatif κ=1,43 → 3,2e-15 ; racine double : Δ = 0 calculé sans arrondi → Tol = 0 ;
    //   Δ ≈ 0 : κ=1,9e8 → 4,2e-7 (on ne garde que ~√ε : le PROBLÈME est mal conditionné).
    @CsvSource(textBlock = """
            # a,      b,      c,                 RA r1,              RA r2,              Tol,   diagnostic
              1,      -3,     2,                 1.0,                2.0,                2e-14, cas nominal
              1,      1e8,    1,                 -99999999.99999999, -1e-8,              5e-15, élimination catastrophique
              1,      1e200,  1,                 -1e200,             -1e-200,            5e-15, b² déborde en +∞
              1e200,  -3e200, 2e200,             1.0,                2.0,                2e-14, ∞ − ∞ = NaN
              1e-200, 1,      1e-200,            -1e200,             -1e-200,            5e-15, absorption
              -2,     3,      5,                 -1.0,               2.5,                4e-15, a négatif
              1,      -2,     1,                 1.0,                1.0,                0,     racine double exacte
              1,      -2,     0.9999999999999999, 0.9999999894632878, 1.0000000105367122, 5e-7,  Δ ≈ 0 mal conditionné
            """)
    @DisplayName("Oracle par référence : N(y, RA) ≤ Tol")
    void parReference(double a, double b, double c, double r1, double r2, double tol, String diagnostic) {
        double[] y = g(a, b, c);                        // y = g(DE)
        double[] ra = (r1 == r2) ? new double[]{r1} : new double[]{r1, r2};
        assertEquals(ra.length, y.length, "nombre de racines");
        for (int i = 0; i < ra.length; i++) {
            double n = erreurRelative(y[i], ra[i]);     // N
            assertTrue(n <= tol,                        // verdict : N ≤ Tol (faux si y = NaN)
                    "racine " + i + " : y = " + y[i] + ", RA = " + ra[i] + ", N = " + n + " > Tol = " + tol);
        }
    }

    // ---------------------------------------------------------------
    // 2. Cas particuliers du contrat (exception attendue = PASS)
    // ---------------------------------------------------------------
    @Test
    @DisplayName("a = 0 : IllegalArgumentException attendue")
    void aNul() {
        assertThrows(IllegalArgumentException.class, () -> g(0, 2, 1));
    }

    @ParameterizedTest(name = "[{index}] ({0}, {1}, {2}) : Δ < 0 → aucune racine réelle")
    @CsvSource({"1, 0, 1", "1, 1, 1", "2, -4, 2.01"})
    @DisplayName("Δ < 0 : tableau vide")
    void discriminantNegatif(double a, double b, double c) {
        assertEquals(0, g(a, b, c).length);
    }

    // ---------------------------------------------------------------
    // 3. Oracles SANS résultat attendu
    // ---------------------------------------------------------------

    /** Résidu comparé à l'échelle des termes (erreur inverse) : |a·r² + b·r + c| ≤ 8·ε·(|a|r² + |b·r| + |c|). */
    @ParameterizedTest(name = "[{index}] ({0}, {1}, {2})")
    @CsvSource({"1, 1e8, 1", "3, 7, 2", "2, -4, 1.99"})
    @DisplayName("Propriété : résidu (erreur inverse)")
    void residu(double a, double b, double c) {
        for (double r : g(a, b, c)) {
            double res = Math.abs(a * r * r + b * r + c);
            double ech = Math.abs(a) * r * r + Math.abs(b * r) + Math.abs(c);
            assertTrue(res <= 8 * EPS * ech, "r = " + r + " : résidu " + res + " > " + 8 * EPS * ech);
        }
    }

    /** Viète : r1 + r2 = −b/a et r1·r2 = c/a. */
    @ParameterizedTest(name = "[{index}] ({0}, {1}, {2})")
    @CsvSource({"1, 1e8, 1", "3, 7, 2"})
    @DisplayName("Propriété : relations de Viète")
    void viete(double a, double b, double c) {
        double[] r = g(a, b, c);
        assertThat(r[0] + r[1]).isCloseTo(-b / a, withinPercentage(1e-10));
        assertThat(r[0] * r[1]).isCloseTo(c / a, withinPercentage(1e-10));
    }

    /** racines(k·a, k·b, k·c) = racines(a, b, c) avec k = 2^10 : produit exact, donc égalité stricte. */
    @ParameterizedTest(name = "[{index}] ({0}, {1}, {2}) et ×1024")
    @CsvSource({"3, 7.1, 2", "1, 1e8, 1", "-2, 3, 5"})
    @DisplayName("Métamorphique : changement d'échelle")
    void echelle(double a, double b, double c) {
```

```
        double k = 1024;                                // 2^10 : produit exact
        assertArrayEquals(g(a, b, c), g(k * a, k * b, k * c)); // strict
    }

    /** racines(a, −b, c) = −racines(a, b, c) ; les racines sont triées : l'opposé inverse l'ordre. */
    @ParameterizedTest(name = "[{index}] ({0}, {1}, {2}) et ({0}, -{1}, {2})")
    @CsvSource({"3, 7.1, 2", "1, 1e8, 1"})
    @DisplayName("Métamorphique : opposé")
    void oppose(double a, double b, double c) {
        double[] r = g(a, b, c);
        double[] s = g(a, -b, c);
        assertEquals(-r[0], s[1], 0.0);
        assertEquals(-r[1], s[0], 0.0);
    }

    /** On choisit les racines, puis on fabrique l'équation (x − r1)(x − r2) = 0. */
    @ParameterizedTest(name = "[{index}] racines choisies {0} et {1}")
    @CsvSource({"-1e8, -1e-8", "1, 2", "-3, 1e-5"})
    @DisplayName("Oracle par construction")
    void construction(double r1, double r2) {
        double[] r = g(1, -(r1 + r2), r1 * r2);
        assertThat(r[0]).isCloseTo(Math.min(r1, r2), withinPercentage(1e-9));
        assertThat(r[1]).isCloseTo(Math.max(r1, r2), withinPercentage(1e-9));
    }

    // ------------------------------------------------------------------
    // 4. Combinaisons : un tableau de valeurs par paramètre, JUnit exécute
    //    TOUS les cas possibles (produit cartésien : 3 × 4 × 3 = 36 cas).
    // ------------------------------------------------------------------
    static final double[] VALEURS_A = {-2, 1, 3};
    static final double[] VALEURS_B = {-5, 0, 1, 1e8};
    static final double[] VALEURS_C = {-4, 0, 2};

    static Stream<Arguments> toutesLesCombinaisons() {
        return Arrays.stream(VALEURS_A).boxed().flatMap(a ->
                Arrays.stream(VALEURS_B).boxed().flatMap(b ->
                        Arrays.stream(VALEURS_C).mapToObj(c -> Arguments.of(a, b, c))));
    }

    /** Aucun RA : nombre de racines donné par le signe EXACT de Δ (BigDecimal), puis résidu (erreur inverse). */
    @ParameterizedTest(name = "[{index}] a = {0}, b = {1}, c = {2}")
    @MethodSource("toutesLesCombinaisons")
    @DisplayName("Combinaisons de paramètres : tous les cas possibles")
    void combinaisons(double a, double b, double c) {
        BigDecimal A = new BigDecimal(a), B = new BigDecimal(b), C = new BigDecimal(c);
        int signeDelta = B.multiply(B).subtract(A.multiply(C).multiply(BigDecimal.valueOf(4))).signum();
        double[] r = g(a, b, c);
        assertEquals(signeDelta + 1, r.length, "nombre de racines");   // Δ<0 → 0, Δ=0 → 1, Δ>0 → 2
        for (double x : r) {
            double res = Math.abs(a * x * x + b * x + c);
            double ech = Math.abs(a) * x * x + Math.abs(b * x) + Math.abs(c);
            assertTrue(res <= 8 * EPS * ech, "x = " + x + " : résidu " + res + " > " + 8 * EPS * ech);
        }
    }

    /**
     * Cas de test GÉNÉRÉS à l'exécution (tests dynamiques JUnit 5) : oracle par construction.
     * Graine fixe : les 20 cas sont toujours les mêmes, un échec se reproduit.
     * Exposants distincts : racines bien séparées, donc κ petit et Tol = 10⁻⁹ % largement justifiée.
     */
    @TestFactory
    @DisplayName("Tests dynamiques : 20 équations générées par construction")
    Stream<DynamicTest> constructionGeneree() {
        Random alea = new Random(563);                  // graine fixe
        return IntStream.rangeClosed(1, 20).mapToObj(i -> {
            int e1 = alea.nextInt(-8, 9);
            int e2 = e1 + 1 + alea.nextInt(0, 8);       // e2 > e1
            double r1 = (alea.nextBoolean() ? 1 : -1) * (1 + alea.nextInt(9)) * Math.pow(10, e1);
            double r2 = (alea.nextBoolean() ? 1 : -1) * (1 + alea.nextInt(9)) * Math.pow(10, e2);
            return dynamicTest("cas " + i + " : racines choisies " + r1 + " et " + r2, () -> {
                double[] r = g(1, -(r1 + r2), r1 * r2);
                assertThat(r[0]).isCloseTo(Math.min(r1, r2), withinPercentage(1e-9));
                assertThat(r[1]).isCloseTo(Math.max(r1, r2), withinPercentage(1e-9));
            });
        });
    }

    /** Bonus : référence BigDecimal à 50 chiffres (la formule naïve n'y souffre plus d'élimination). */
    @ParameterizedTest(name = "[{index}] ({0}, {1}, {2})")
    @CsvSource({"1, 1e8, 1", "3, 7.1, 2", "1e-3, -5, 1e3"})
    @DisplayName("Oracle différentiel (BigDecimal 50 chiffres)")
    void differentiel(double a, double b, double c) {   // a > 0 : (−b + √Δ)/2a est la plus grande racine
        MathContext mc = new MathContext(50);
```

```
        BigDecimal A = new BigDecimal(a), B = new BigDecimal(b);
        BigDecimal C = new BigDecimal(c), DEUX = BigDecimal.valueOf(2);
        BigDecimal s = B.multiply(B)
                .subtract(A.multiply(C).multiply(BigDecimal.valueOf(4)))
                .sqrt(mc);
        BigDecimal ref = B.negate().add(s).divide(A.multiply(DEUX), mc);
        assertThat(g(a, b, c)[1]).isCloseTo(ref.doubleValue(), withinPercentage(1e-12));
    }
}
```

D. src/test/java/inf563/solveur/SolveurStableTest.java

```java
package inf563.solveur;

import org.junit.jupiter.api.DisplayName;

/** Toute la suite contre la version stable : tout doit être VERT. */
@DisplayName("Solveur STABLE (formule stable + mise à l'échelle)")
class SolveurStableTest extends SuiteOraclesSolveur {

    @Override
    double[] g(double a, double b, double c) {
        return Solveur.racines(a, b, c);
    }
}
```

E. src/test/java/inf563/solveur/SolveurNaifTest.java

```java
package inf563.solveur;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Tag;

/**
 * Toute la suite contre la version naïve : certains tests DOIVENT échouer.
 * C'est le but du TP : remplir le tableau « l'oracle détecte / ne détecte pas ».
 *
 * <p>Exclue de « mvnw test » (tag naif) ; à lancer avec « mvnw test -Pnaif »
 * ou directement depuis le Test Explorer de VS Code.
 */
@Tag("naif")
@DisplayName("Solveur NAÏF (formule du lycée)")
class SolveurNaifTest extends SuiteOraclesSolveur {

    @Override
    double[] g(double a, double b, double c) {
        return Solveur.racinesNaif(a, b, c);
    }
}
```

F. pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- INF563 - Fil rouge A : solveur d'équation du second degré (séances 2 et 3) -->
    <groupId>inf563</groupId>
    <artifactId>solveur-second-degre</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <junit.version>5.13.4</junit.version>
        <assertj.version>3.27.3</assertj.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.junit</groupId>
                <artifactId>junit-bom</artifactId>
```

```
                <version>${junit.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <dependencies>
        <!-- JUnit 5 (Jupiter) + tests paramétrés (@ParameterizedTest, @CsvSource) -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <scope>test</scope>
        </dependency>
        <!-- AssertJ : isCloseTo(..., within / withinPercentage) -->
        <dependency>
            <groupId>org.assertj</groupId>
            <artifactId>assertj-core</artifactId>
            <version>${assertj.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.14.0</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.3</version>
                <configuration>
                    <!-- Par défaut on n'exécute que la version stable (suite verte).
                         La version naïve (qui DOIT échouer) se lance avec : mvnw test -Pnaif -->
                    <excludedGroups>${groupes.exclus}</excludedGroups>
                    <groups>${groupes.inclus}</groups>
                </configuration>
            </plugin>
        </plugins>
    </build>

    <profiles>
        <profile>
            <id>defaut</id>
            <activation>
                <activeByDefault>true</activeByDefault>
            </activation>
            <properties>
                <groupes.exclus>naif</groupes.exclus>
                <groupes.inclus></groupes.inclus>
            </properties>
        </profile>
        <profile>
            <id>naif</id>
            <properties>
                <groupes.exclus></groupes.exclus>
                <groupes.inclus>naif</groupes.inclus>
            </properties>
        </profile>
    </profiles>
</project>
```

G. .vscode/settings.json et .mvn/wrapper/maven-wrapper.properties

```
{
    "java.configuration.updateBuildConfiguration": "automatic",
    "java.compile.nullAnalysis.mode": "disabled",
    "maven.executable.preferMavenWrapper": true,
    "files.encoding": "utf8"
}
```

```
wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3.9.11-bin.zip
```